



IT3010

Network Design and Management

Lecture 07
LDAP

Shashika Lokuliyana

Faculty of Computing
Department of CSE



SLIIT

Discover Your Future

Today's lecture overview

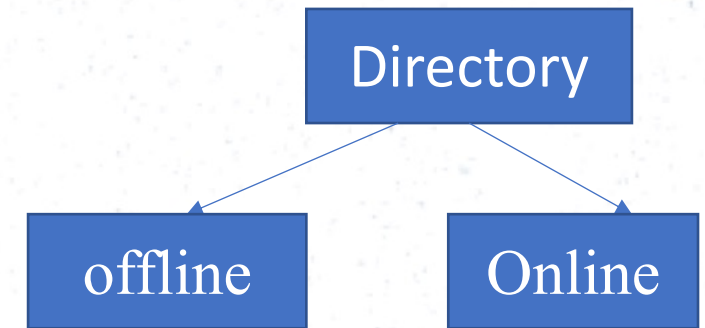
- What is a directory?
- Invention of LDAP.
- Understanding LDAP.
 - The LDAP Models.
 - The LDAP Information Model.
 - The LDAP Naming Model.
 - The LDAP Functional Model.
 - The LDAP Security Model.
- LDAP Data Interchange Format (LDIF).
- Schema Design.
- Namespace Design
- Topology Design
- Replication Design

Reference Book

- Understanding and Deploying LDAP Directory Service (2nd Edition) by Timothy A. Howes, Mark C. Smith and Gordon S. Good.

What is a directory?

- In brief, a directory is a specialized database.
- Most people are familiar with various kinds of directories, whether they realize or not.
- Everyday examples of directories:
 - Phone books (white and yellow pages)
 - TV guide
 - Shopping catalogs
 - Library card catalogs etc.
- These everyday directories are known as offline directories.
 - They are static: Doesn't update/ change often. Maybe yearly.



What is a directory? Contd.

- But in computing/ networking world we have online directories.
- Online directories are dynamic.
 - Offline directories are relatively static. They do not change often. By contrast online directories can be kept much more up-to-date.
- Online directories are flexible.
 - Offline directories are static in terms of their content. Cost of adding new information to printed (offline) directories is huge (cost of re-designing, re-printing and re-distributing). By contrast online directories can be easily extended with new types of information.
- Online directories can be made secure.
 - Offline directories offer little, if any, security. By contrast online directories centralize information allowing access to that information to be controlled.

What is a directory? Contd.

- Online directories can be personalized.
 - TV guide and phone book are personalized in a regional basis.
 - Everyone access the same card catalog at the library.
 - But online directories can easily provide more personalized views of the directory for each user.
 - By identifying the users who access the directory and storing profile information about them.

Directories

- Size of the information.
 - Good for storing small pieces of information, not multi MB files.
 - Good at storing pointers to large things, than the large things themselves.
- Character of the information.
 - Good at storing attribute based information where you can break your information into name-value pairs.
- Read-to-write ratio.
 - Best for information that are read far more than it is written.
 - If more writes, think about using a DB instead.
- Search capability.
 - Directories are designed to make searching fast.
 - If your application requires fast searching, directory maybe a good choice.
- Standards-based access.
 - If you need standards-based access to your information (for example, access by a different vendors software which is using the same standard access protocol).

What is LDAP?

- Stands for Lightweight Directory Access Protocol.
- A standard, extensible Internet Protocol used to access directory services.
- Lightweight alternative to DAP.
- Uses TCP/IP instead of OSI stack.



The origins and history of LDAP

- X.500
 - World's first standard directory service and the specification.
 - Jointly developed by International Telecommunication Union (ITU-T) and International Organization for Standardization (ISO).
 - First published in early 1990s.
 - Designers expectation was: One day, eventually there would be one fully interconnected global directory service, the Directory.

X.500 Contd.

- Advantages:
 - Extensibility
 - Rich set of search operations
 - A highly distributed system
 - Open standard: Not controlled by any vendor or tied to any OS
- Disadvantages:
 - Highly complex specification and difficult to implement (as a result, many early versions of X.500 software were buggy)
 - Rely on a large suite of protocols (DAP, DSP, DOP and DISP)
 - Based on OSI network model.

X.500 Contd.

- Since the designers of the X.500 believed that eventually the OSI model will replace the TCP/IP model which never happened, unfortunately.
- One of the best known implementations of X.500 specification was *Quipu*, developed by UCL which was free and open source.

The origins and history of LDAP Contd.

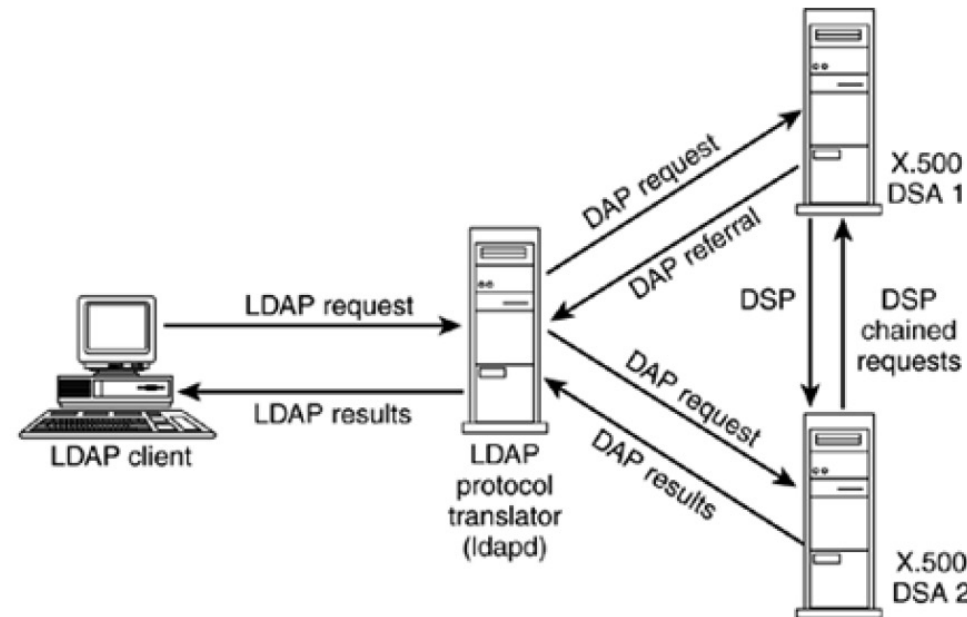
- DAP (Directory Access Protocol): X.500's directory client access protocol (i.e. the protocol that client's used to communicate with the X.500 server)
- Had some issues
 - Large.
 - Complex.
 - Difficult to implement.
 - Most implementations performed poorly.
- Two alternatives were proposed,
 - DAS (Directory Assistance Service) – RFC 1202.
 - DIXIE (Directory Interface to X.500 Implemented Efficiently) – RFC 1249.
- Both proved to be better than DAP.
- But, both had one shortcoming: Both were closely tied with *Quipu*.

The birth of LDAP

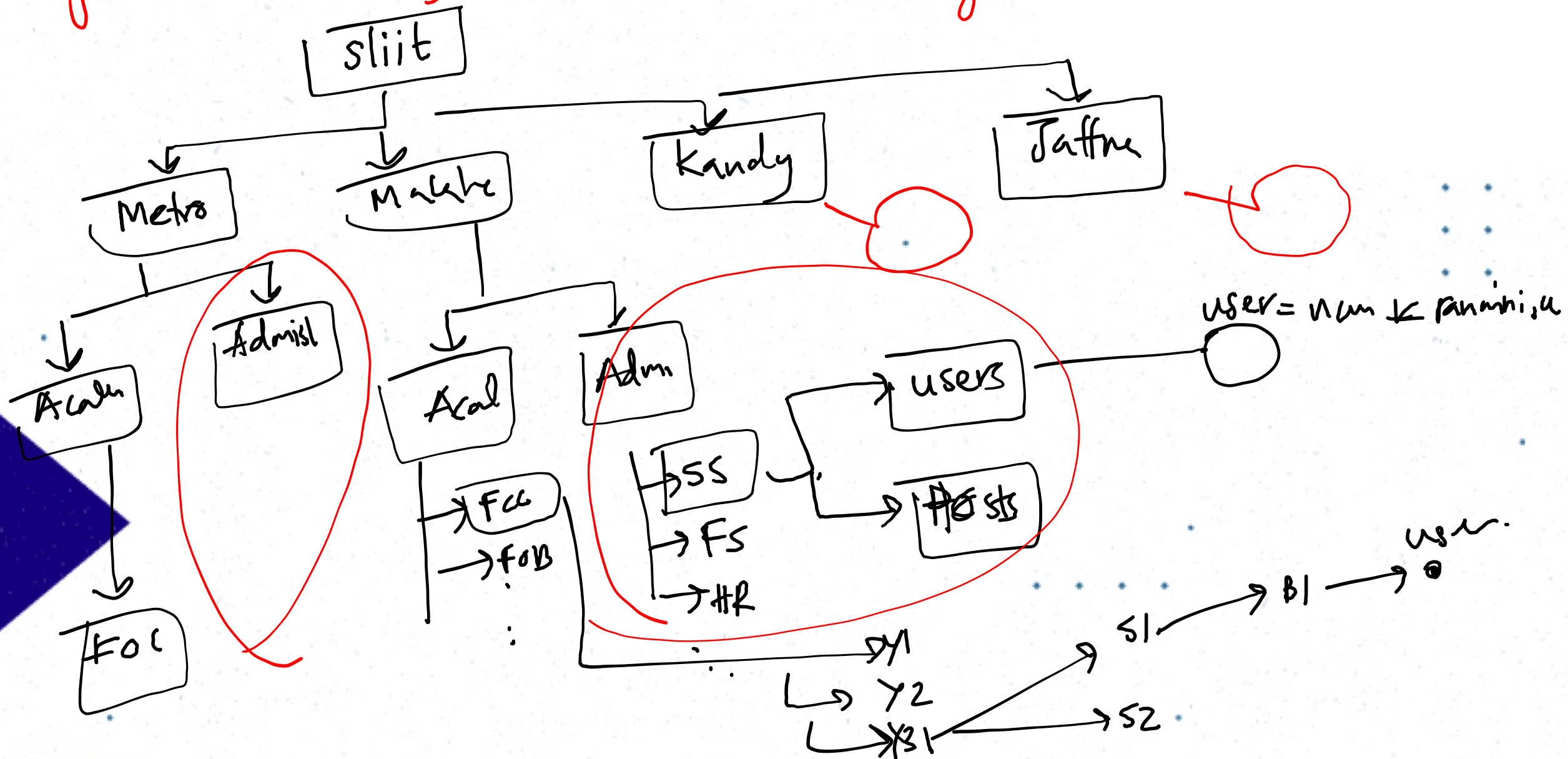
- *With the initiation taken by DAS and DIXIE towards a lightweight access protocol to X.500 directory service, IETF decided to produce a standard, full-featured, lightweight directory access protocol.*
- *As a result Lightweight Directory Access Protocol (LDAP) was born.*
- One key advantage: LDAP was fully based on TCP/IP stack rather than the OSI stack.
- During the initial stages LDAP was exclusively used to provide a front end to X.500 directory services.
- Initial implementation of LDAP was done at University of Michigan and it came to be known as the U-M LDAP implementation.

U-M LDAP

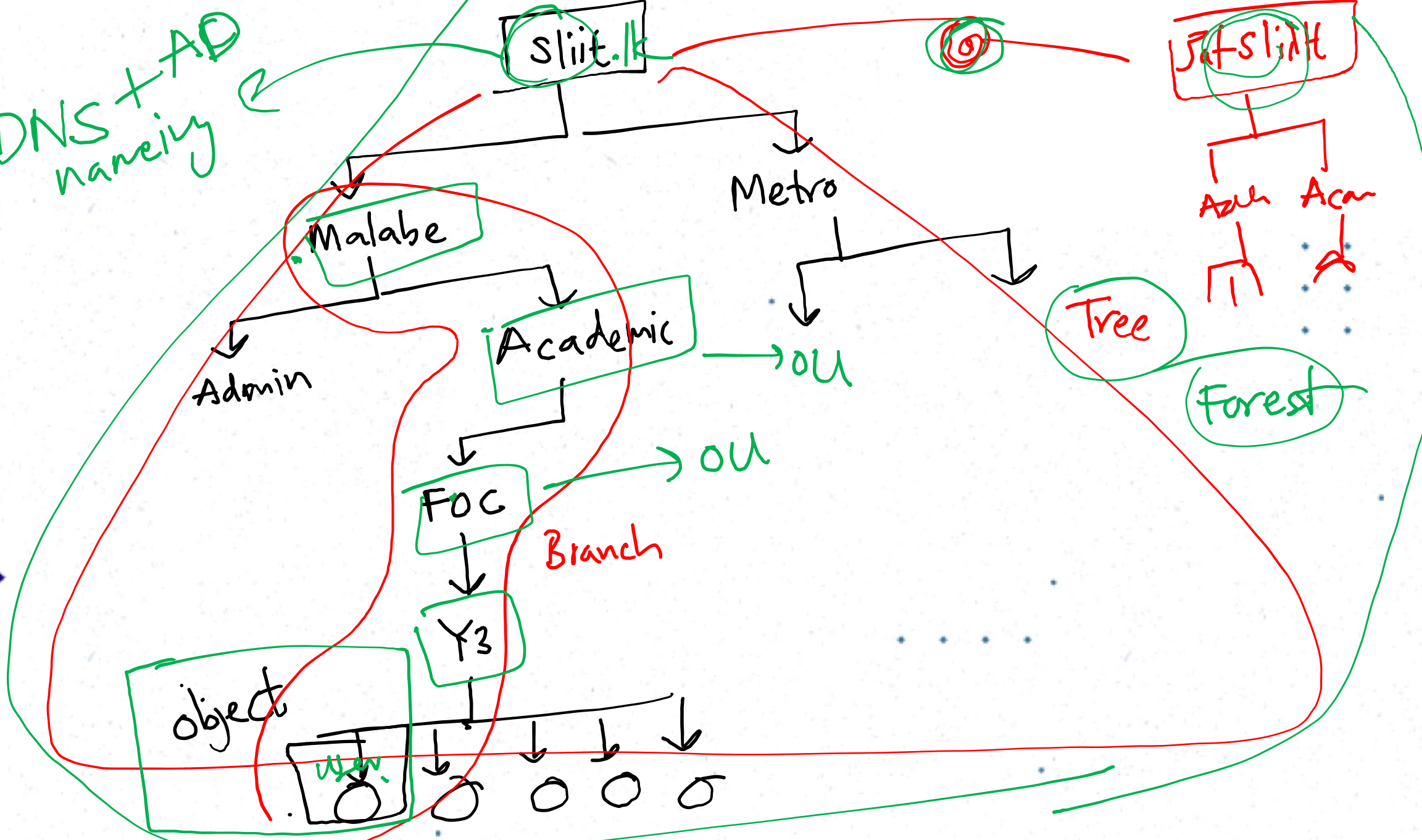
- The key server side component is *ldapd*, which is a LDAP-to-X.500 DAP protocol translator.
- Although LDAP solved part of the problem, still, the server implementation of the X.500 directory was large, complex and difficult to deploy.

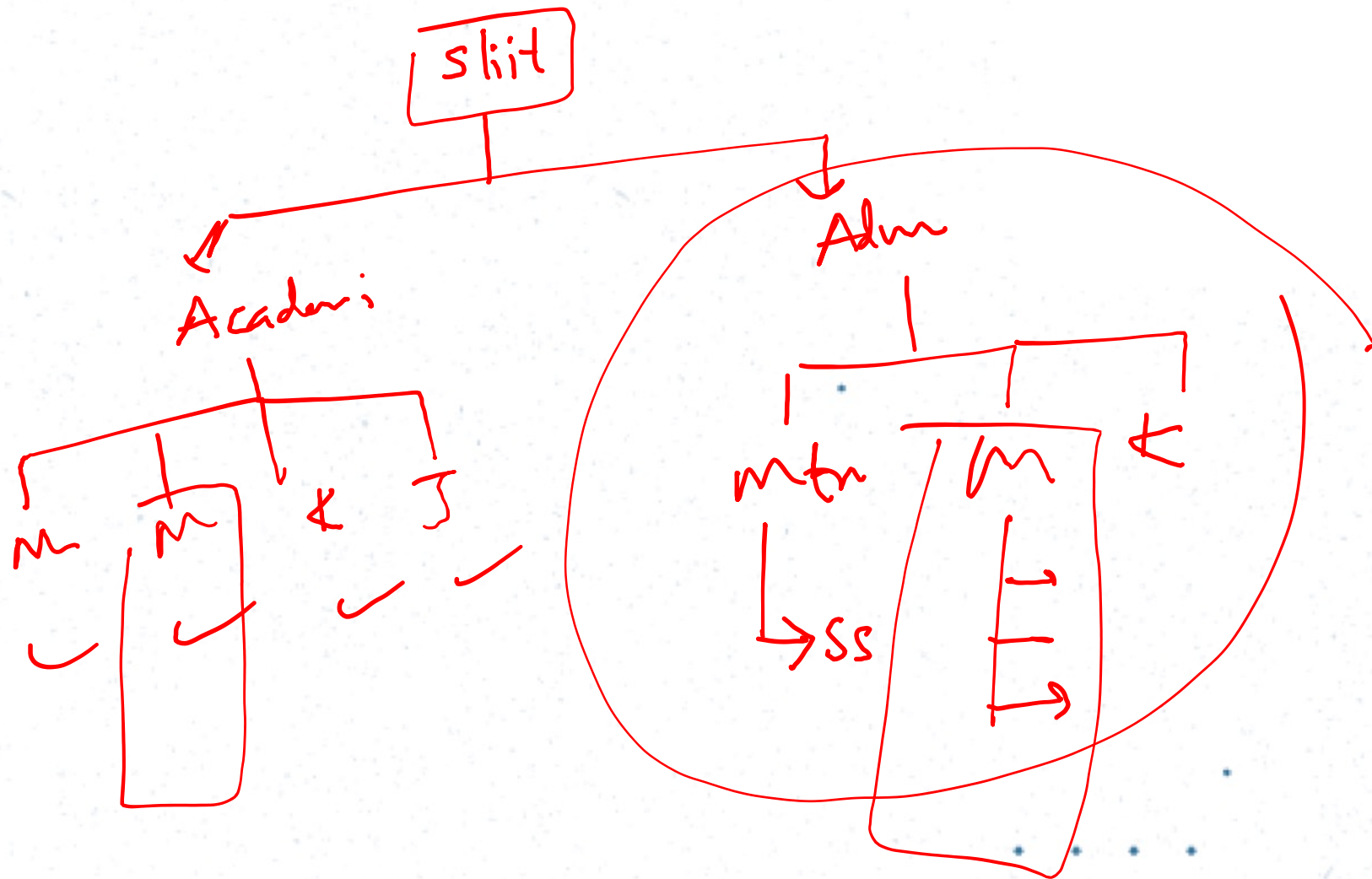


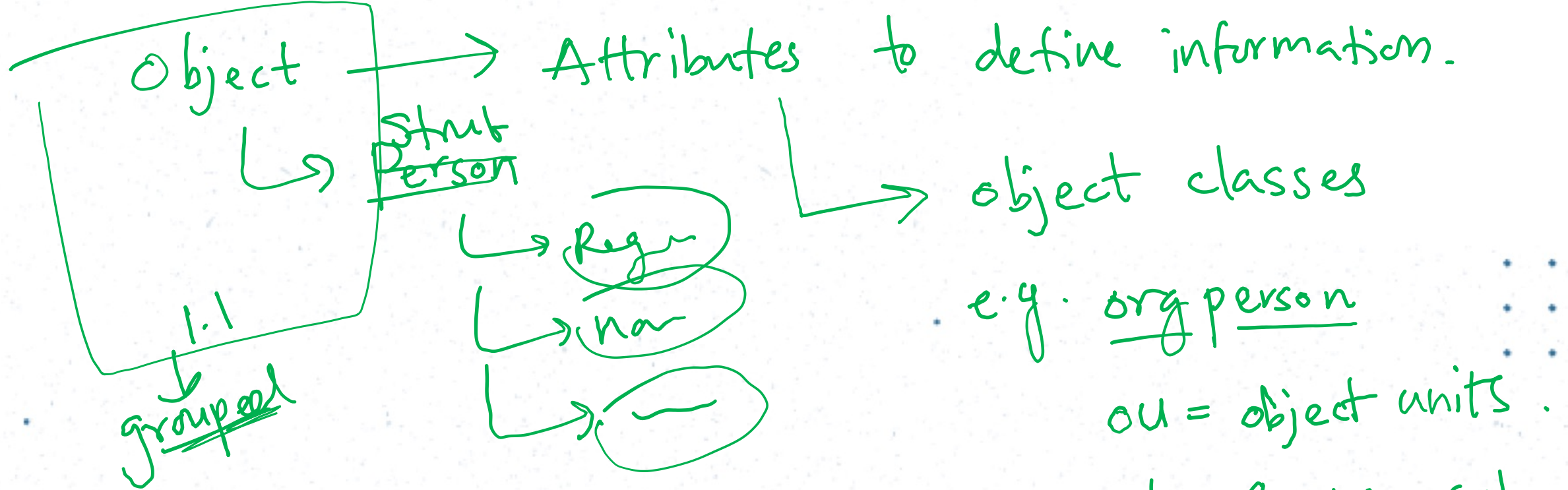
organization Directory Structure → organization structure.



DNS + AD
naming







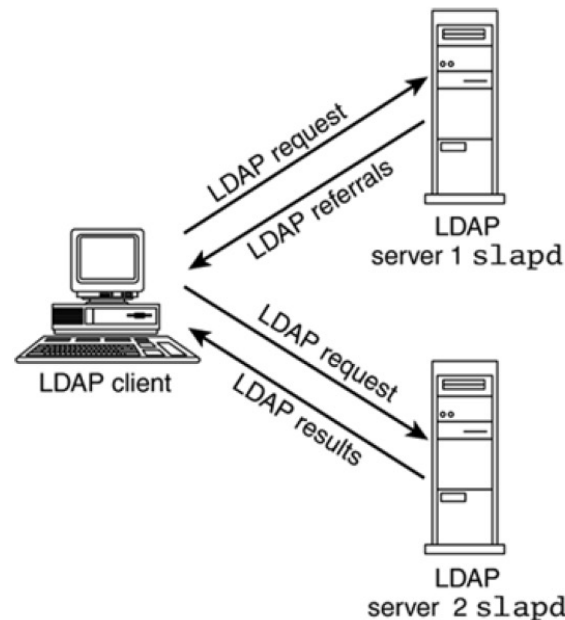
* share resource inside a network the only method to share resource is groups





Standalone LDAP (slapd)

- As a solution designers thought of keeping the best features of X.500 and developing LDAP as a standalone directory service specification.
- *As a result today's standalone LDAP service came to the role.*



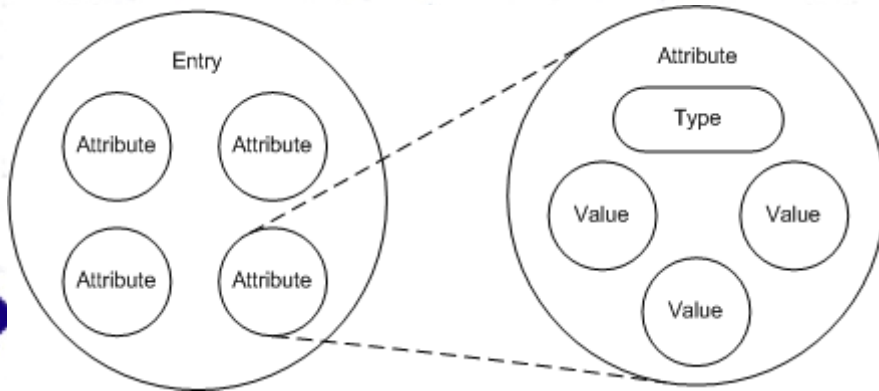
General purpose LDAP directory service implementations

- Microsoft Active Directory
- IBM Directory Server
- Oracle Internet Directory
- OpenLDAP slapd server

LDAP MODELS

- Information
 - Structure of information stored in an LDAP directory.
- Naming
 - How information is organized and identified.
- Functional operations
 - Describes what operations can be performed on the information stored in an LDAP directory.
- Security
 - Describes how the information can be protected from unauthorized access.

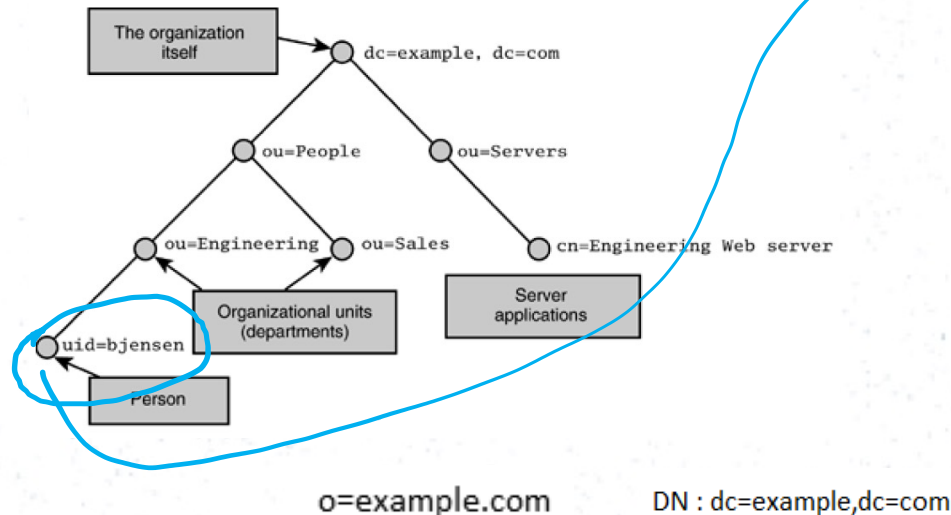
LDAP Information Model



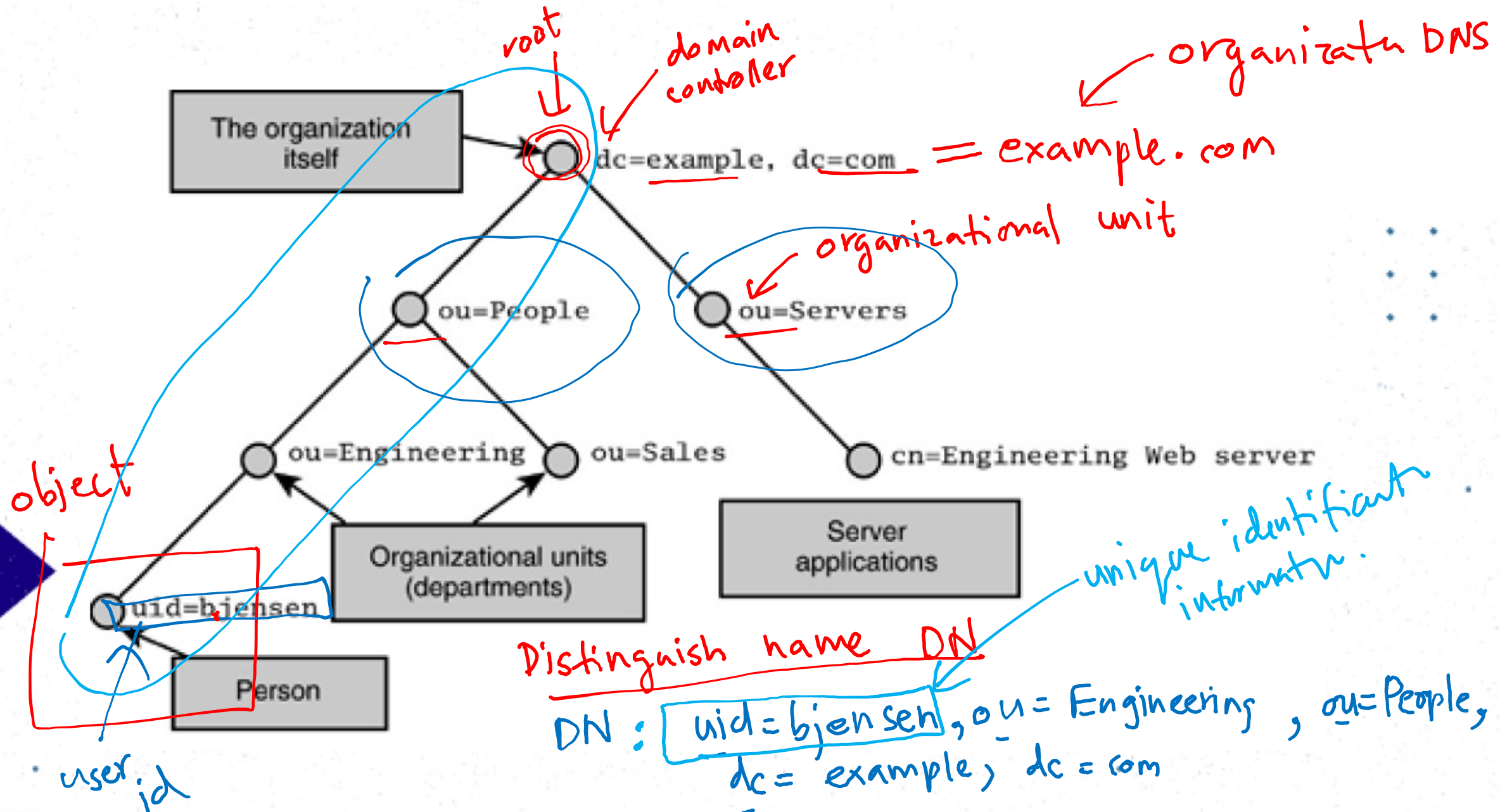
- The LDAP information model defines the types of data and basic units of information that you can store in your directory.
- The basic unit of information in the directory is the entry.
 - A collection of information about an object (for example, real world objects like people, departments, servers, printers etc.).
- An entry is composed of a set of attributes.
- Each of which describes one particular characteristic of the object.
- Each attribute has a type and one or more values.

LDAP Information Model contd.

- Each directory has a Distinguished Name (DN).
- Ex: Organization shown in the following figure has the DN dc=example, dc=com



Attribute type	Attribute values
cn:	Barbara Jensen Babs Jensen
sn:	Jensen
telephoneNumber:	+1 408 555 1212
mail:	babs@example.com



DN: cn=kamal peries, ou=BS,
ou=user, dc=ndm, dc=lk

dc=ndm, dc=lk

hierarchical Topology.

object

attributes

cn=kamal peries

regist = IT...
email = amal.p@my.sliit.lk
GPA =

ou=user

ou=BS

ou=BS

cn=kamal peries

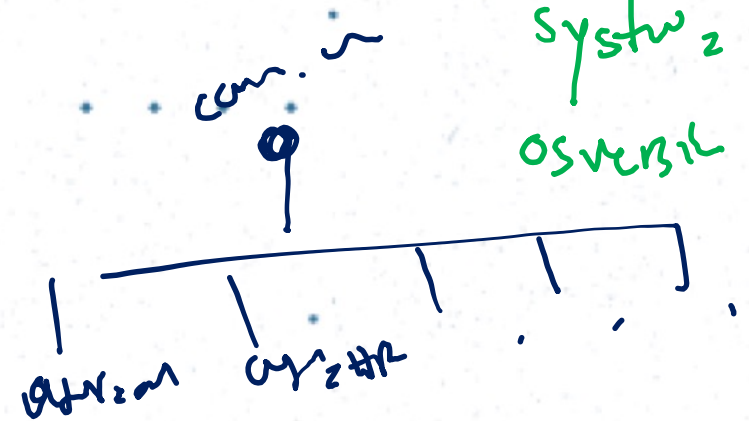
ou=BS

ou=BS

uid=sashika-lab

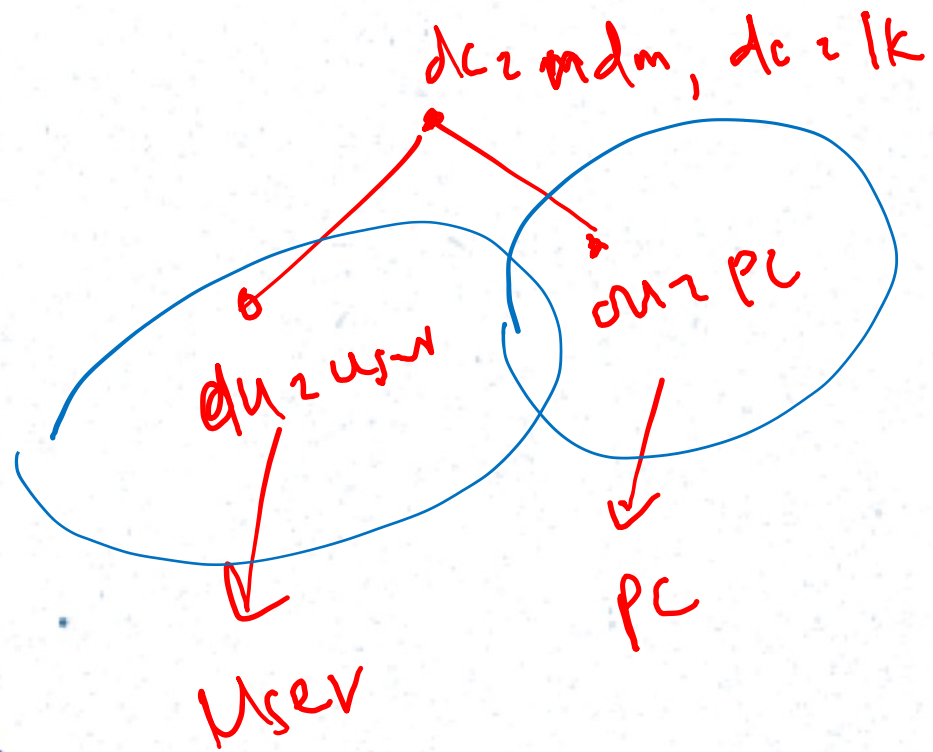
uid=My-laptop

domain =
hostname =
system =
osversion =



e.g. slit student naming ~~email~~ schemas for AD/LDAP
registration no @my.slit.lk

staff slit → firstname . first letter
of last
name @slit.lk
shashika . l @



LDIF file

~~##~~ domain

dn: dc2ndm, dc2lk

dc2ndm

dc2lk

~~##~~ user object

dn: ou2user, dc2ndm, dc2lk

o: organization

~~##~~ PC object

dn:

o: .

LDAP Information Model contd.

- Attributes can be classified as user attributes and operational attributes
 - *User attributes*, the "normal" attributes of an entry, may be modified by the users of the directory (with appropriate permissions).
 - *Operational attributes* are special attributes that either modify the operation of the directory server or reflect the operational status of the directory.
- An example of an operational attribute is the `modifyTimeStamp` attribute, which reflects the time that the entry was last modified and is automatically maintained by the directory.
- When an entry is sent to a client, operational attributes are not included unless the client requests them by name.
- The collections of all information relating to attributes are called the *directory schemas*.

LDAP Information Model contd.

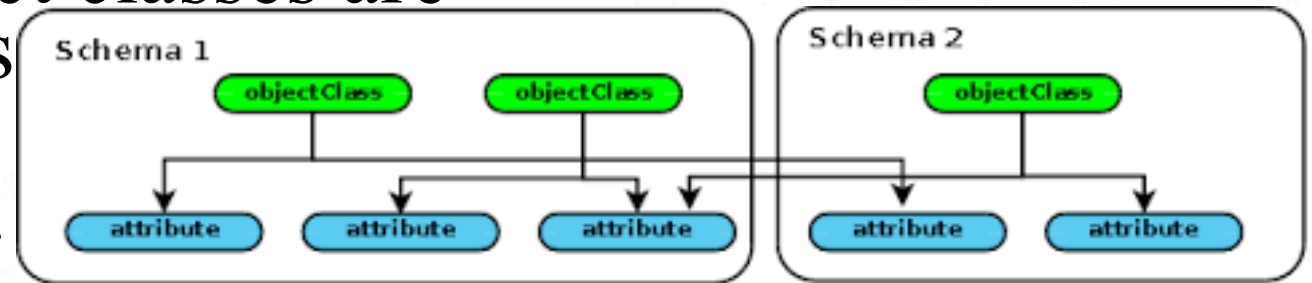
- LDAP Schemas

- When you create an entry in a DIT its data contents are contained in attributes, which are grouped into objectclasses, which are packaged into schemas.

- Schemas define what object classes are allowed. (An objectClass is

- Where they are stored.
 - What attributes they have.
 - What attributes are optional.
 - Type/ syntax of each attribute.

- LDAP schema must be readable by the client.



LDAP Naming Model

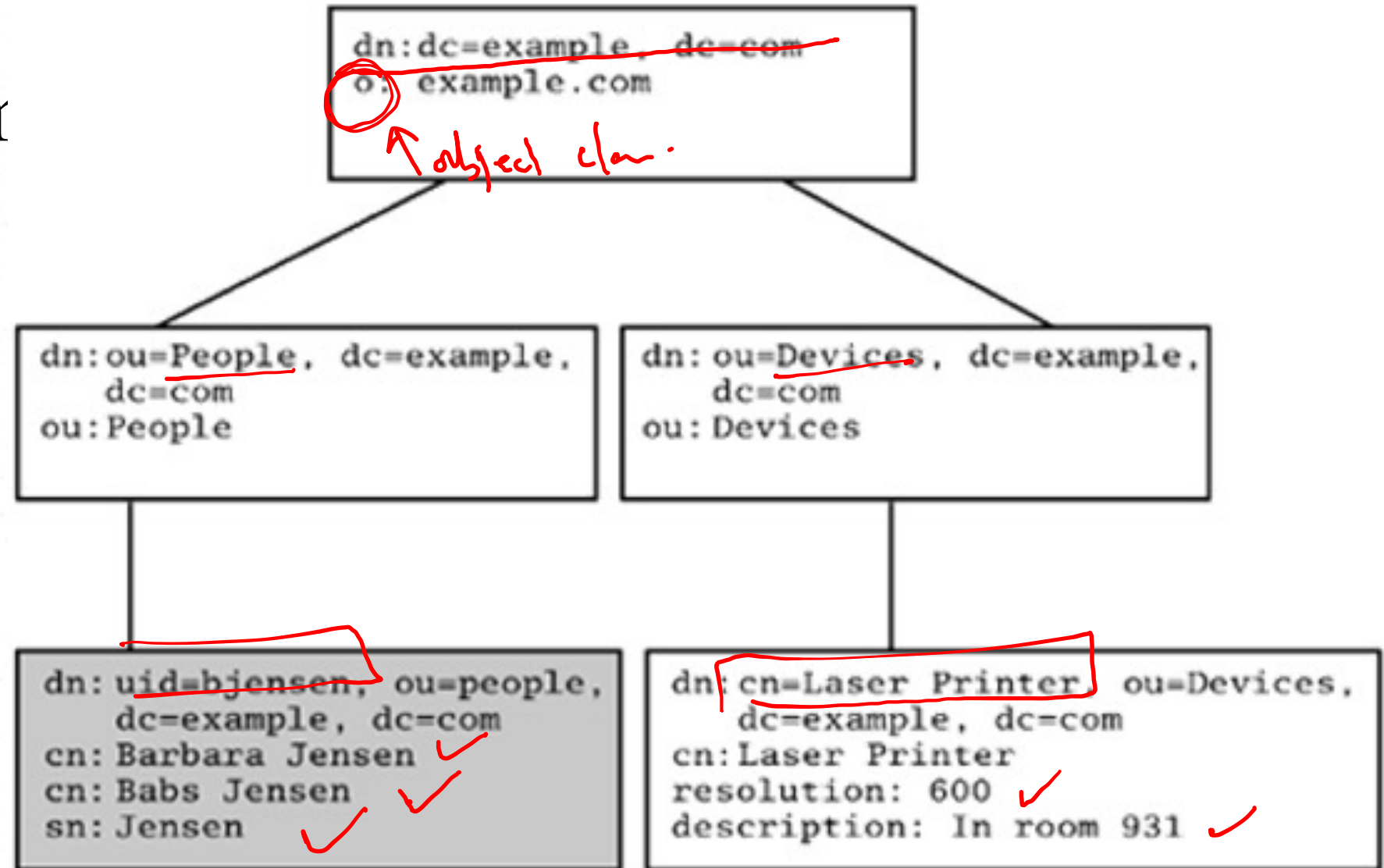
- The LDAP naming model defines how you organize and refer to your data.
- Directory Information Tree (DIT).
 - Usually follows the geographical or organizational scheme.
 - The name of an LDAP entry is formed by connecting, in a series, all the individual names of the parent entries back to the root.
- Distinguished Names (DN).
 - DN is similar to an absolute path of a file within a file systems. DN is the unique identifier of an entry within the DIT. This allows any entry of the DIT to be identified unambiguously. DN of an entry should be unique within the entire DIT.
- Relative Distinguished Names (RDN).
 - RDN is similar to the file name of a file itself within a file system. It is the leftmost component of the DN of an entry. RDNs should be unique only if they share a common immediate parent. This is similar to, you not been able to create two files with the same name within the same folder.

LDAP Naming Example

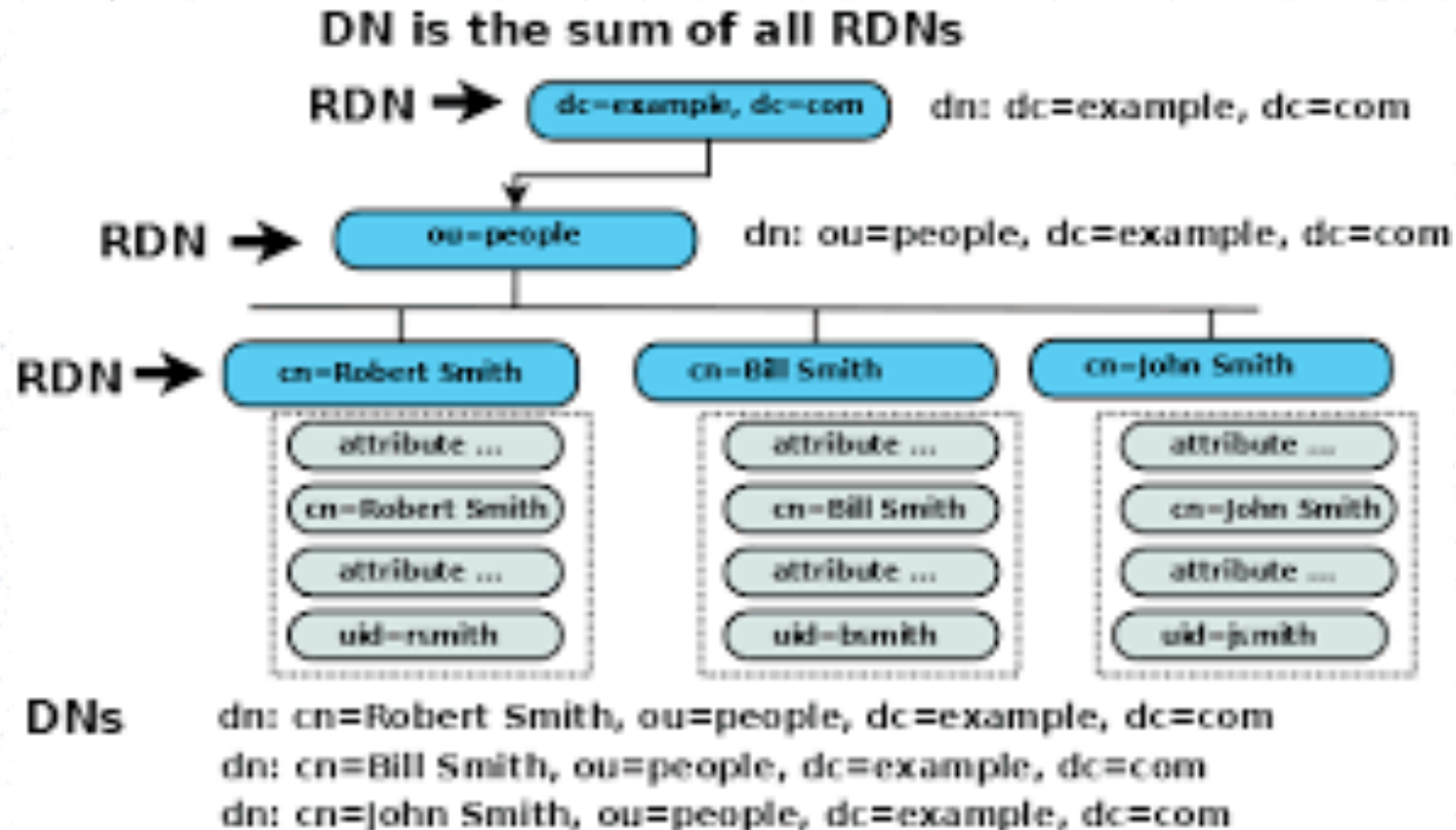
LDAP Naming Examples

Attribute Type	String
CommonName	CN
LocalityName	L
StateorProvinceName	ST
OrganizationName	O
OrganizationalUnitName	OU
CountryName	C
StreetAddress	STREET
domainComponent	DC
Userid	UID

LDAP Naming

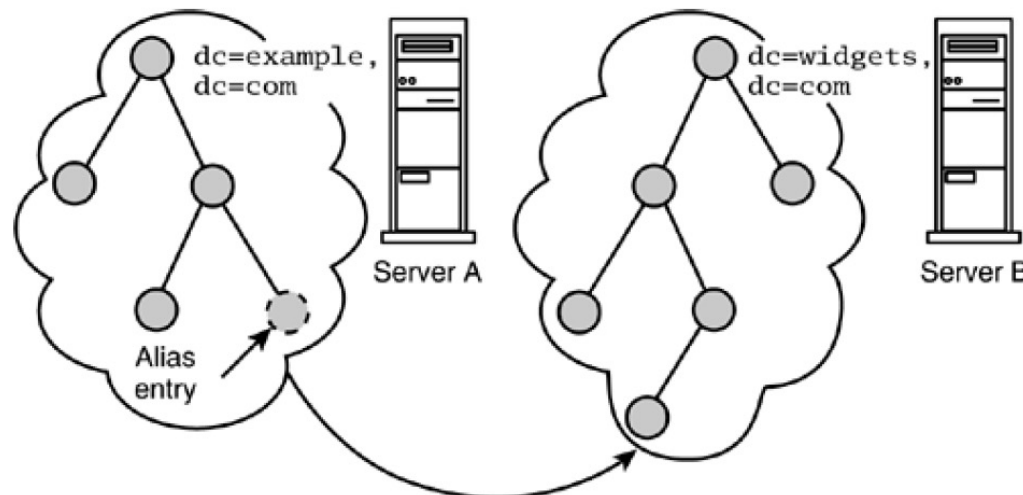


Example



LDAP Naming Model

- Alias entries in the LDAP directory allow one entry to point to another one:
- Similar to symbolic links in UNIX or shortcuts in Windows.
- objectClass=alias
- Not all LDAP implementations support aliases.
- Aliases potentially slow down search operations.



LDAP Functional Model

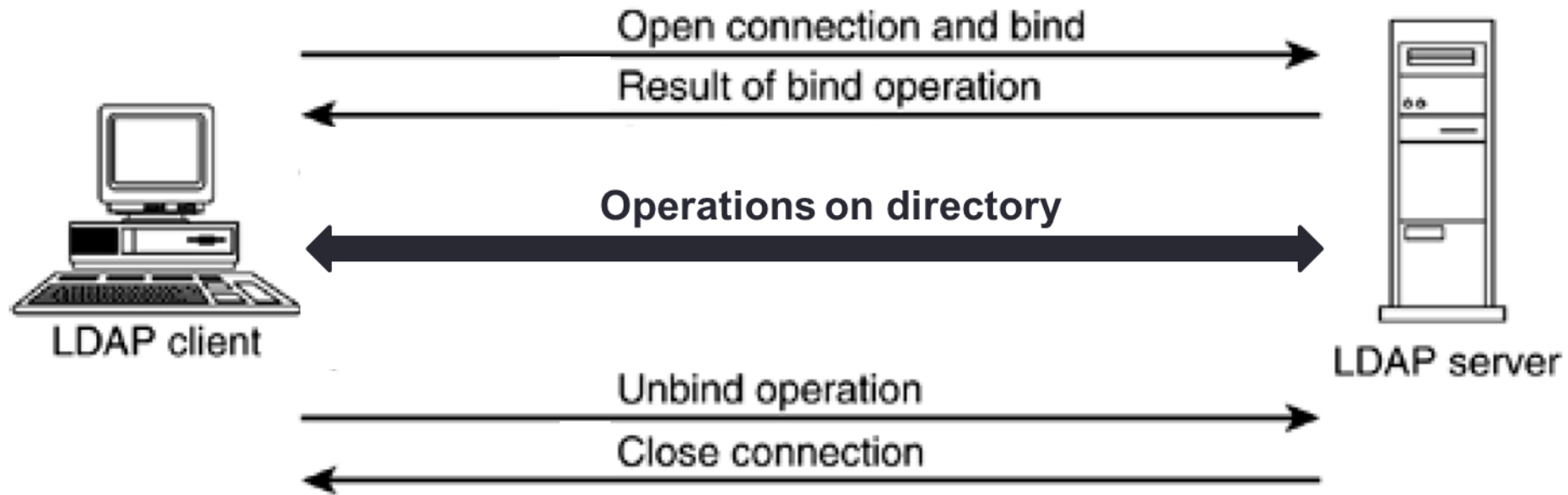
- LDAP functional model consists of a set of operations divided into three groups.
- Authentication and Control operations.
 - Allows the clients to identify themselves to the directory and control some aspects of a session.
 - BIND
 - UNBIND
 - ABANDON
- Interrogation operations.
 - Allows to search the directory and retrieve directory data.
 - Search
 - Compare
- Update operations.
 - Allows to add, delete, rename and change directory entries.
 - Add an entry
 - Delete an entry
 - Modify an entry
 - Rename an entry (i.e. Modify DN/ RDN)



Client/ Server Interaction

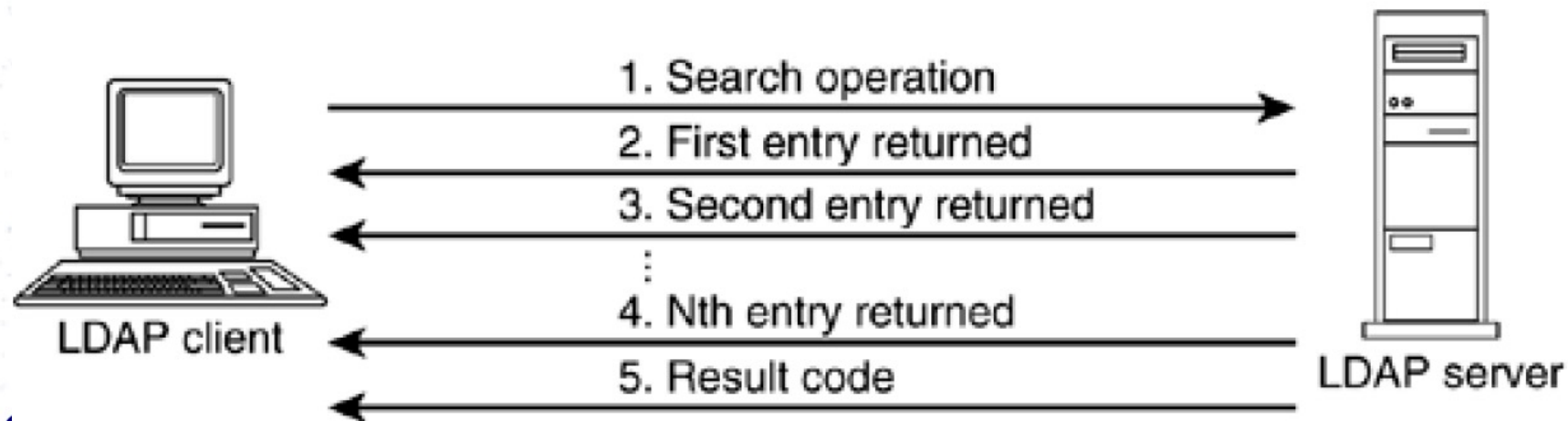
- Client establishes session with server (BIND).
- Client performs operations.
- Client ends the session (UNBIND).

Client/ Server Interaction contd.



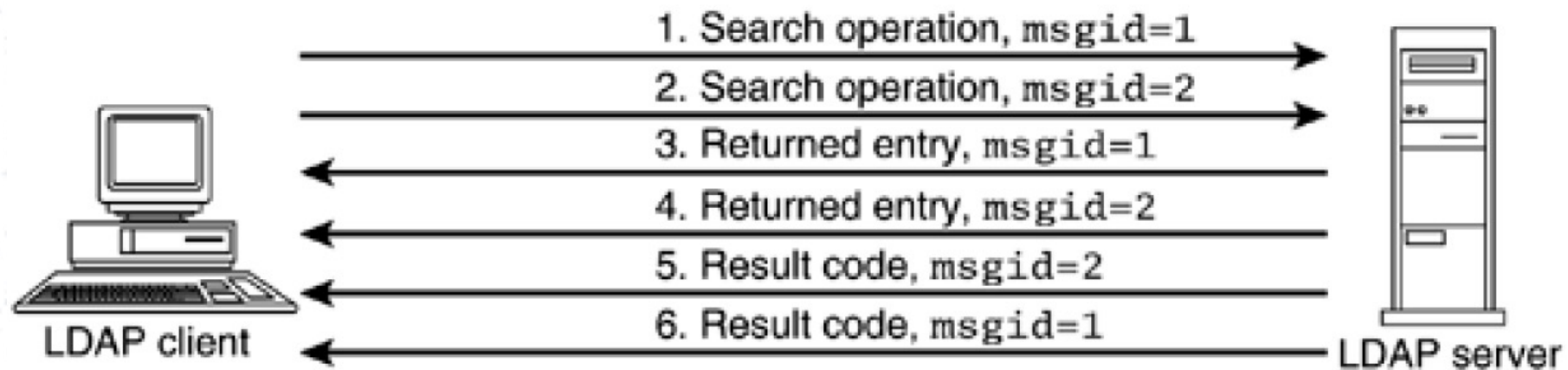
Client/ Server Interaction contd.

- If the client searches the directory and find multiple entries, those entries are sent to the client in a series of LDAP messages, one for each entry.
- Each entry has a unique name called a *Distinguished Name* (DN) which is carried as a text string in the LDAP message.
- Finally the *Result code* is returned: Contains the overall result of the search operation.



Client/ Server Interaction contd.

- LDAP protocol is messages based. Therefore it allows the client to issue multiple requests at once.
 - Client generates a unique message ID for each request.
- Returned results for a specific request are tagged with the request's message ID.
- Allows the client to sort multiple replies to different requests arriving out of order at the same time.

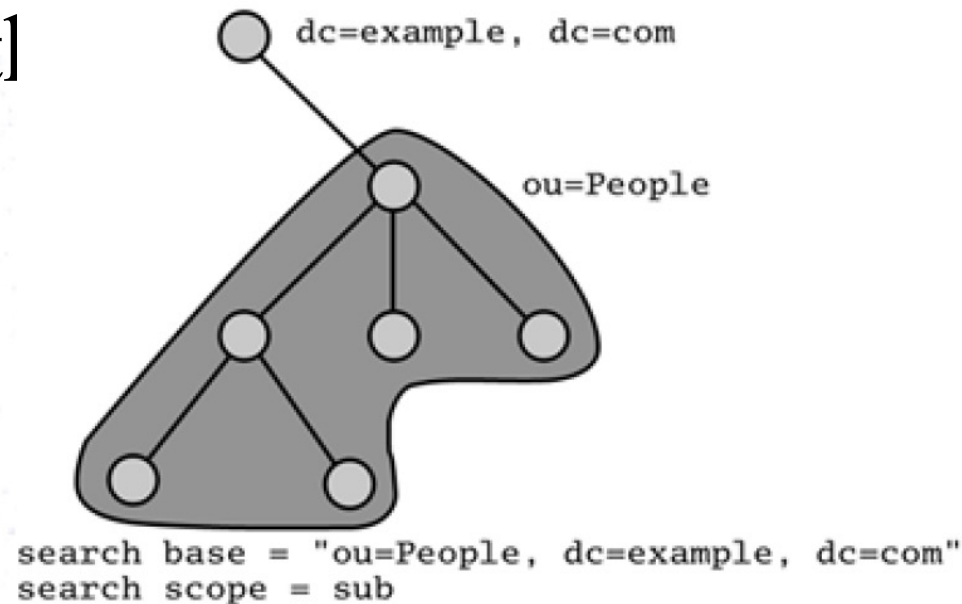


Search Operation

- LDAP search operation provides many parameters that you can use to customize the search operation based on your need. (Page 70 of the reference book).
- There is no specific read operation defined in LDAP. Therefore, you need to customize the search operation using these parameters to act as a read operation.
- One such important parameter in search operation is the *search scope*.
- There are three *search scope* options.

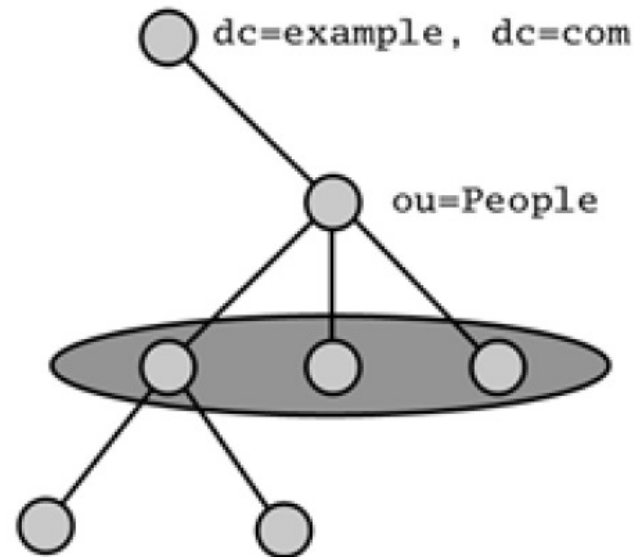
Search operation contd.

- sub
- Search the entire sub tree from the base object all the way down to tl



Search operation contd.

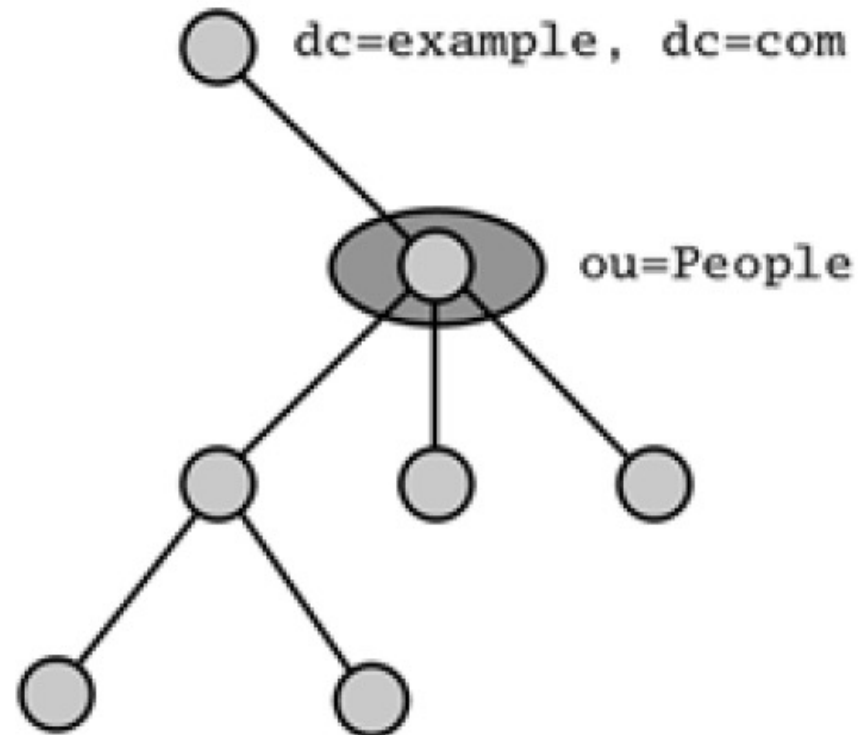
- onelevel
- Search only the immediate children of the entry at the top of the search base



search base = "ou=People, dc=example, dc=com"
search scope = onelevel

Search operation contd.

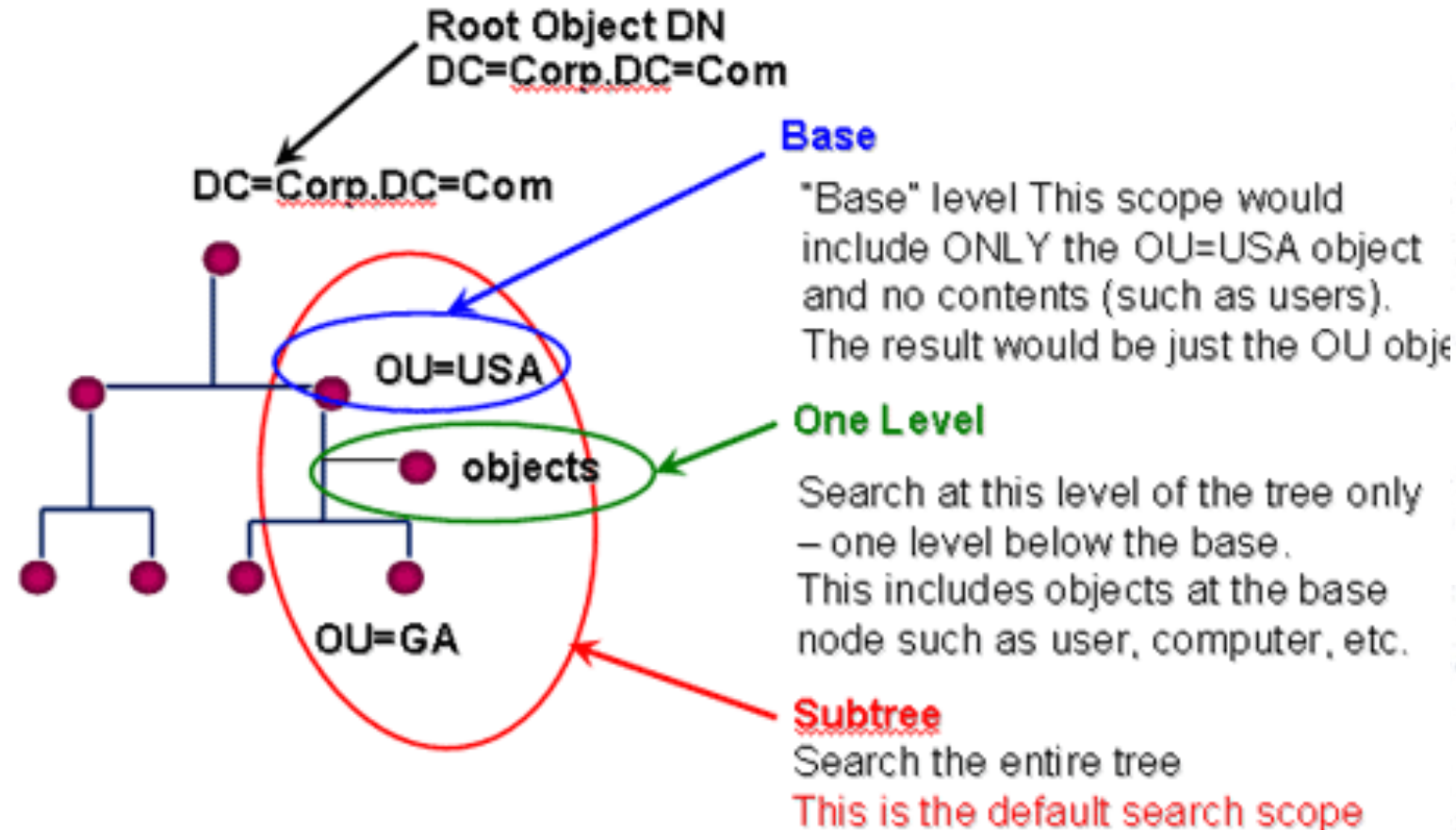
- base
- Limit the search to just the base object



search base = "ou=People, dc=example, dc=com"
search scope = base

Functional Model Interrogation

Search Scope

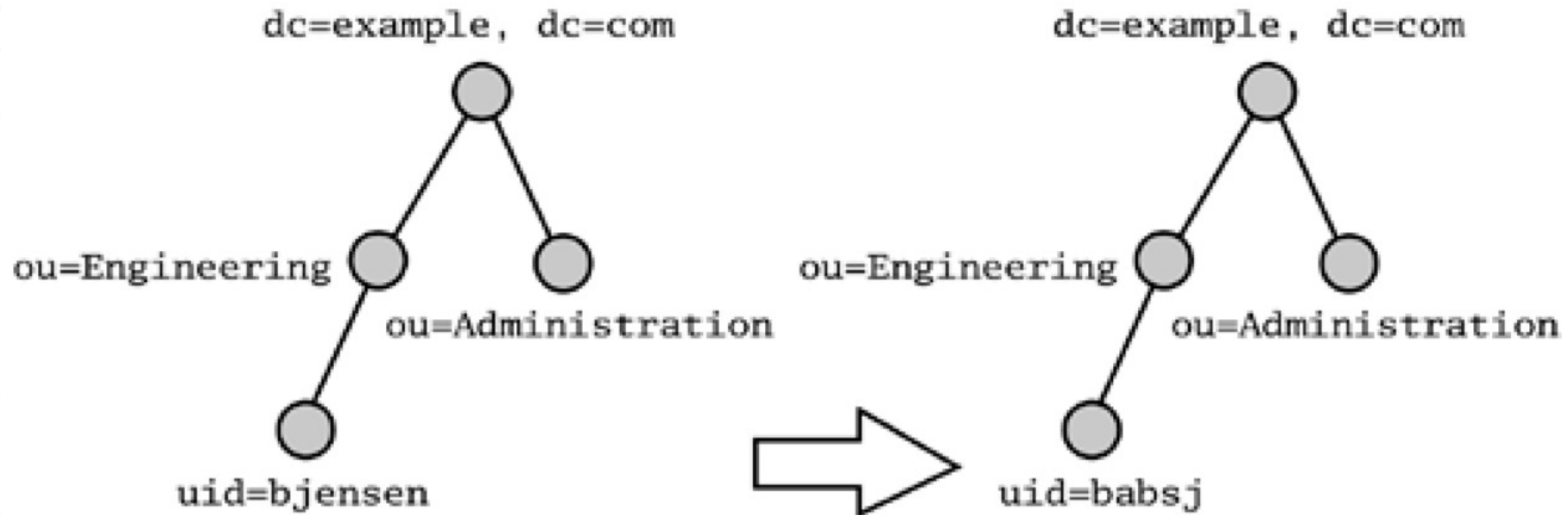


Update Operations

- There are four LDAP update operations
- Add
- Delete
- Rename (Modify DN)
- Modify

Examples: What you can do with LDAP update operations

- Renaming an entry without moving it.

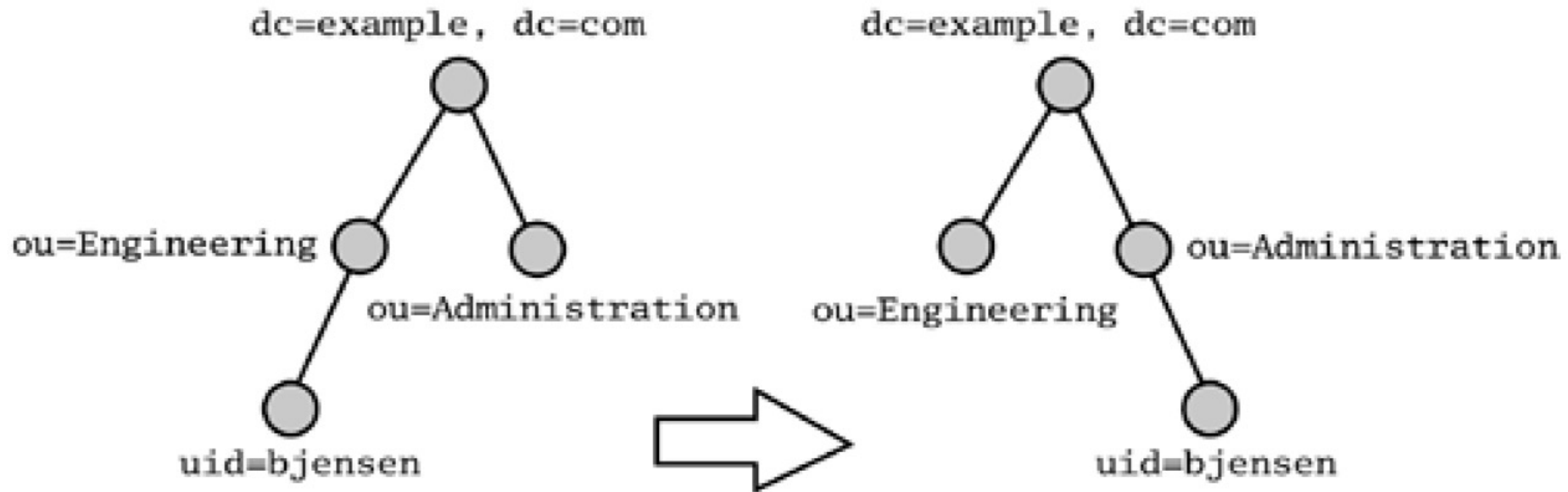


Original DN: uid=bjensen, ou=Engineering, dc=example, dc=com

New DN: uid=babsj, ou=Engineering, dc=example, dc=com

Examples: What you can do with LDAP update operations contd.

- Moving an entry without changing its RDN.

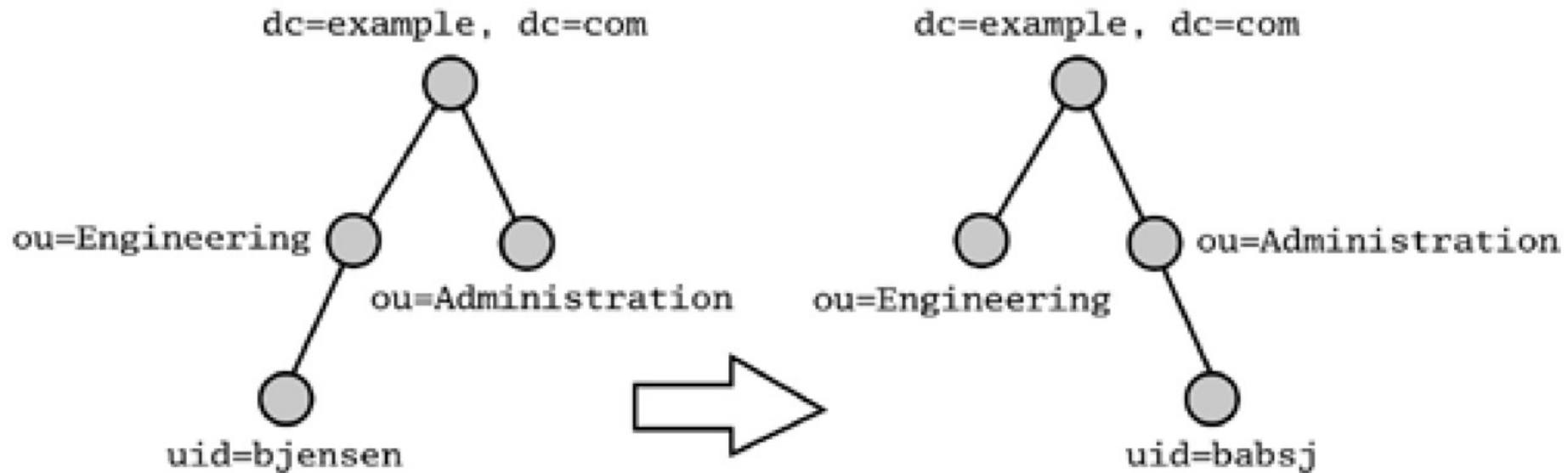


Original DN: `uid=bjensen, ou=Engineering, dc=example, dc=com`

New DN: `uid=bjensen, ou=Administration, dc=example, dc=com`

Examples: What you can do with LDAP update operations contd.

- Moving an entry and changing its RDN simultaneously

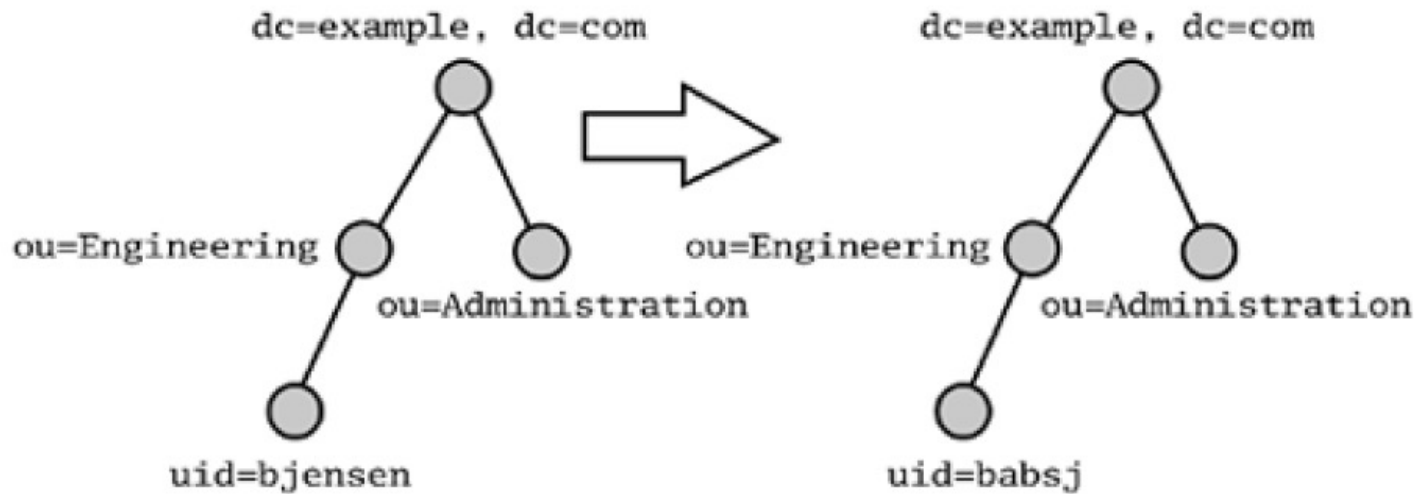


Original DN: `uid=bjensen, ou=Engineering, dc=example, dc=com`

New DN: `uid=babsj, ou=Administration, dc=example, dc=com`

Examples: What you can do with LDAP update operations contd.

- Renaming an entry, deleteoldrdn=true



```
dn:uid=bjensen,  
ou=Engineering,  
dc=example, dc=com
```

```
.  
. .  
. .
```

```
uid: bjensen
```

```
.  
. .  
. .
```

```
dn:uid=babsj,  
ou=Engineering,  
dc=example, dc=com
```

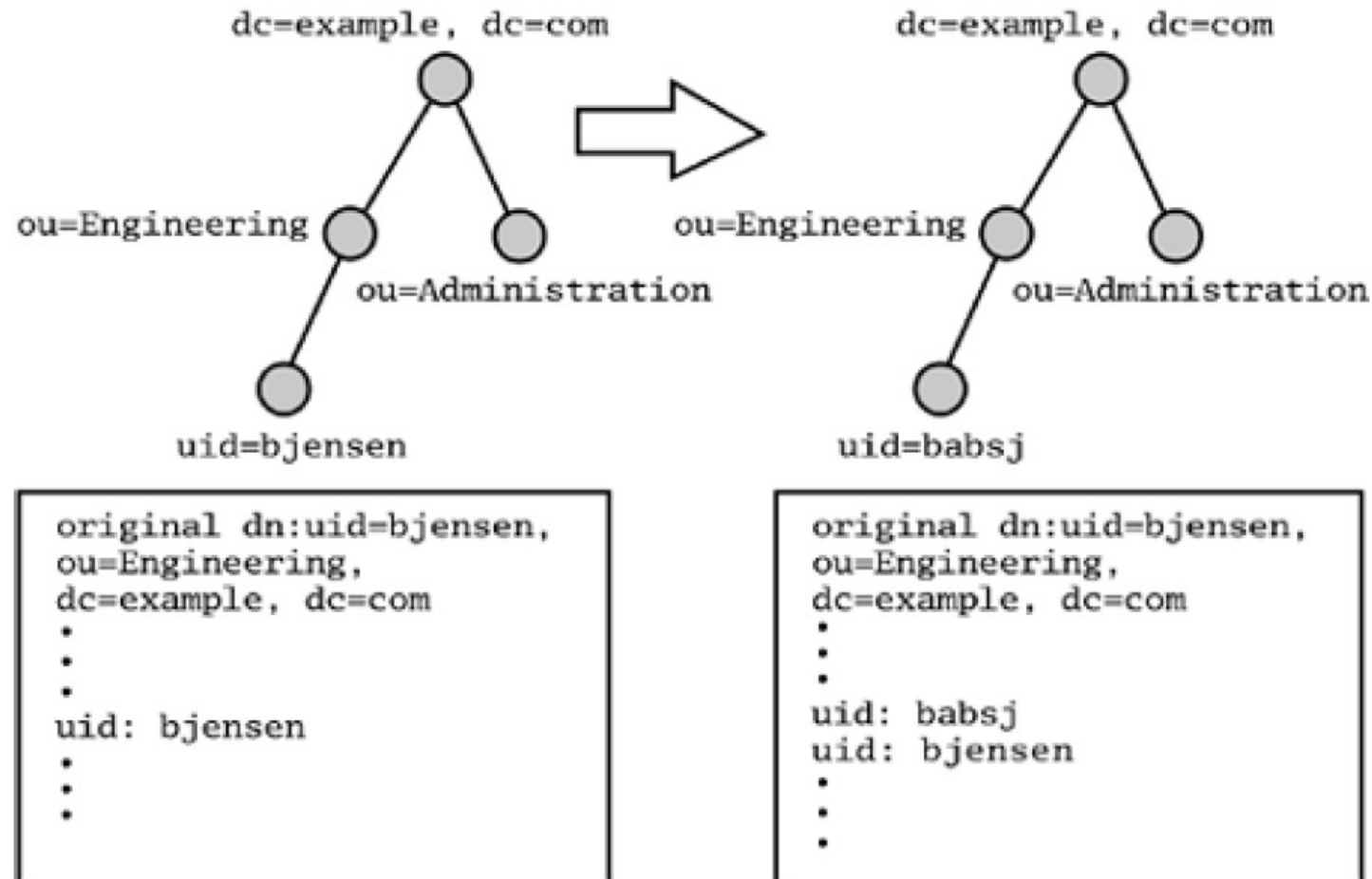
```
.  
. .  
. .
```

```
uid: babsj
```

```
.  
. .  
. .
```

Examples: What you can do with LDAP update operations contd.

- Renaming an entry, deleteoldrdn=false



LDAP Security Model

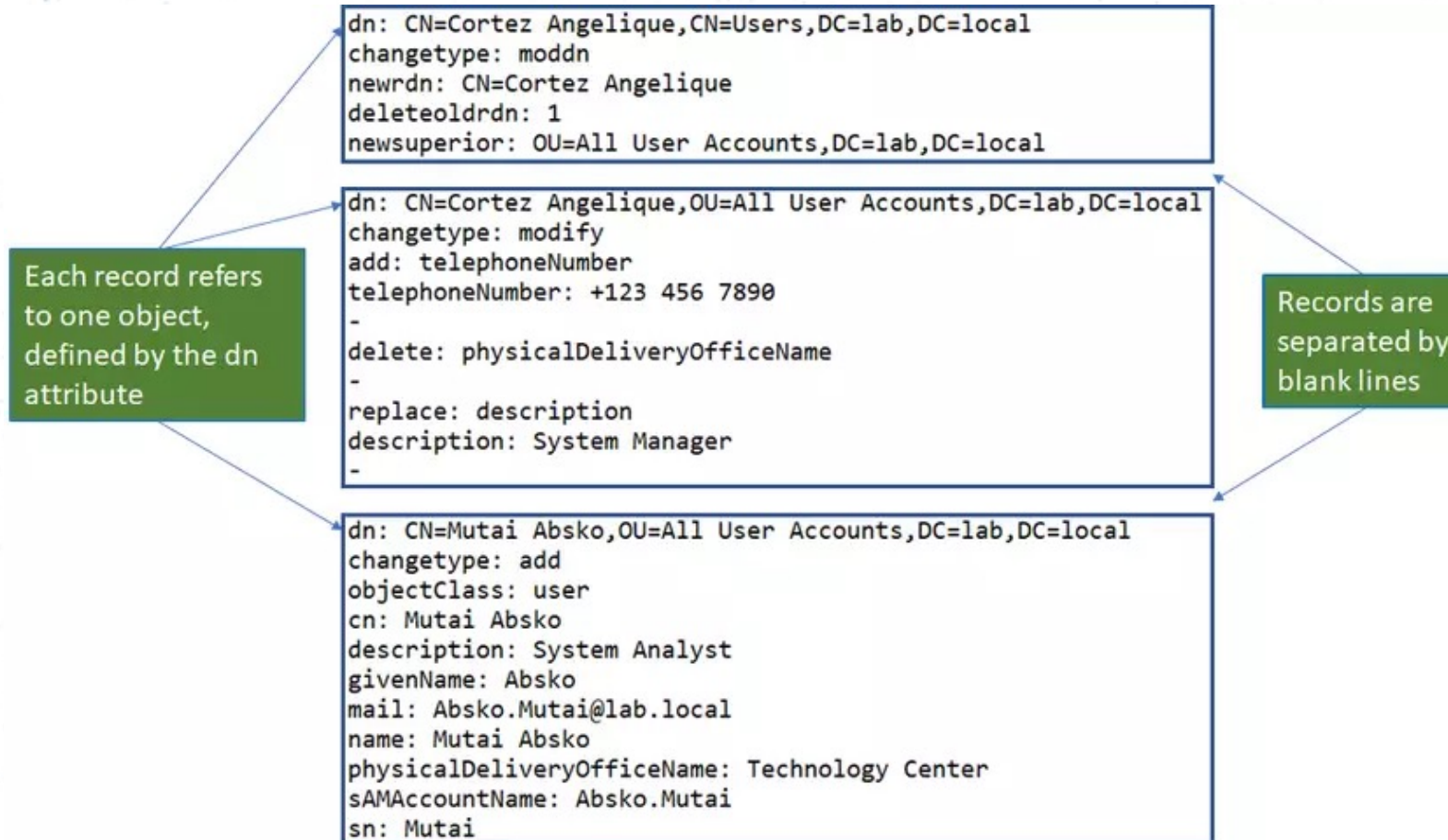
- Security = Encryption + Authentication + Access Control
- No access control in LDAP (yet...).
- Simple authentication (in LDAPv2).
 - DN and password provided.
 - Clear text or BASE 64 encoded.
- Simple authentication over SSL/TLS (in LDAPv3).
- Simple authentication and Security Layer SASL (RFC 2222).
 - Parameters: DN, mechanism, credentials.
 - Provides cross protocol authentication calls.
 - Encryption can be optionally negotiated.
 - `ldap_sasl_bind()` (ver3 call).
 - `ldap://<ldap_server>/?supportedsaslm mechanisms`

LDIF

- Stands for LDAP Data Interchange Format.
- LDIF is a standard text-based format for describing directory entries.
- Defined in RFC 2894.
- LDIF allows to export our directory data and import them to another directory server.
- Only ASCII (LDAPv3 uses UTF-8).
- An LDIF file is
 - A collection of entries separated from each other by blank lines
 - A mapping of attribute names to values
 - A collection of directives that instruct the parser how to process the information
- The data in the LDIF file must obey the schema rules of your LDAP directory

Deploy an LDAP/AD tree in an org
all the objects that
necessary to create
directory structure.

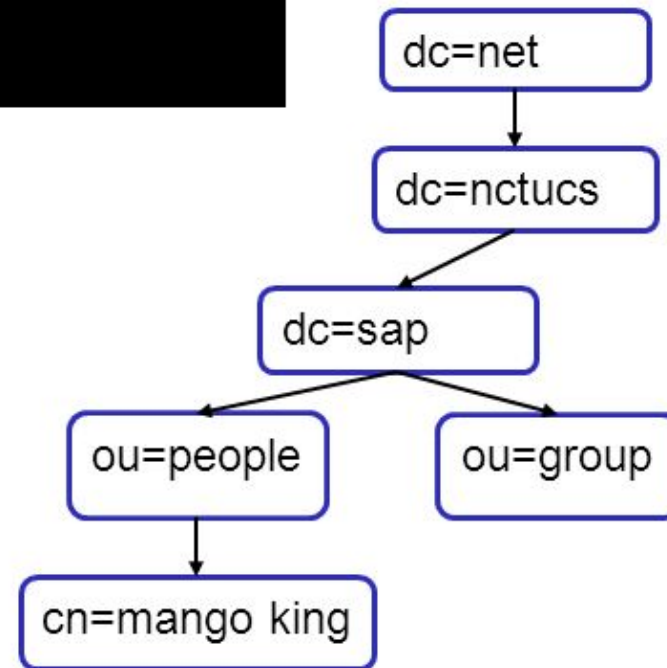
LDIF Format



❑ Sample LDIF

```
# sample entry
dn: cn=mango king,ou=people,dc=sap,dc=nctucs,dc=net
objectClass: person
cn: mango king
sn: king
telephoneNumber: 689-5566
```

dc: domain component
ou: organizational unit
cn: common name
dn: distinguished name
rdn: relative dn



LDIF Example

```
dn: CN=Cox Paul,OU=All User Accounts,DC=labs,DC=local
changetype: add
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: user
cn: Cox Paul
sn: Cox
# This is their job title
description: IT Manager
givenName: Paul
distinguishedName:
  CN=Cox Paul,OU=All User Accounts,DC=labs,DC=local
instanceType: 4
whenCreated: 20190111112006.0Z
whenChanged: 20190904114116.0Z
uSNCreated: 25067
memberOf:
  CN=Professional Services Department,OU=All Groups,DC=labs,DC=local
memberOf: CN=DnsAdmins,CN=Users,DC=labs,DC=local
uSNChanged: 336116
name: Cox Paul
objectGUID: Qxqi/AVGCUyDMSThWIKRHw==
---SNIP---
```

Annotations:

- changetype: add → Changetype
- # This is their job title → Comment
- memberOf: CN=Professional Services Department,OU=All Groups,DC=labs,DC=local
memberOf: CN=DnsAdmins,CN=Users,DC=labs,DC=local → Multi-value attribute
- objectGUID: Qxqi/AVGCUyDMSThWIKRHw== → Encoded value



Schema Design

Schema Design

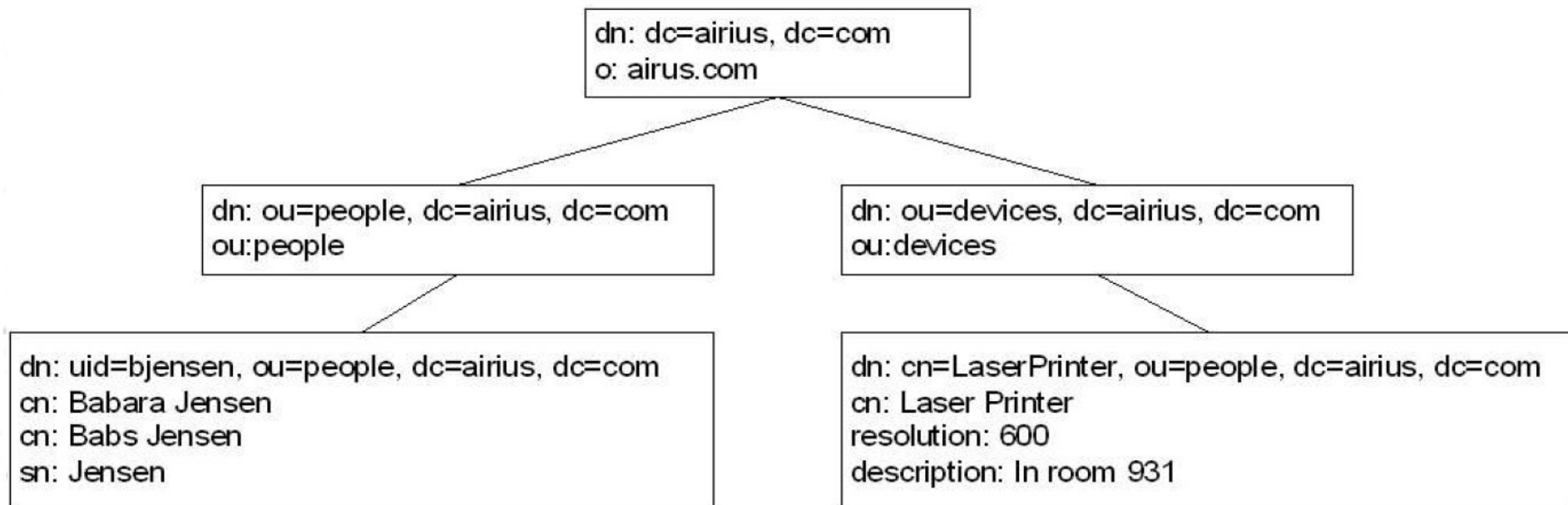
- Similar to database systems, a schema is a set of rules that determines what data can be stored in the directory.
- **Purposes of a schema:**
 - *Ensures that poorly designed applications “play by the rules” and do not store redundant data in the directory.*
 - If every directory-enabled application stored a person's name in a different directory attribute. The result would be wasted storage space.
 - *Can be used to impose constraints on the size, range and format of data values stored in the directory.*
 - Ex: According to the Internet mail standards, e-mail address values should use a restricted set of characters and should conform to a specific format (addr@domain).
 - *Can help slow the effect of directory entropy.*
 - Up to a certain level schema rules help in preventing disorder within your directory service.

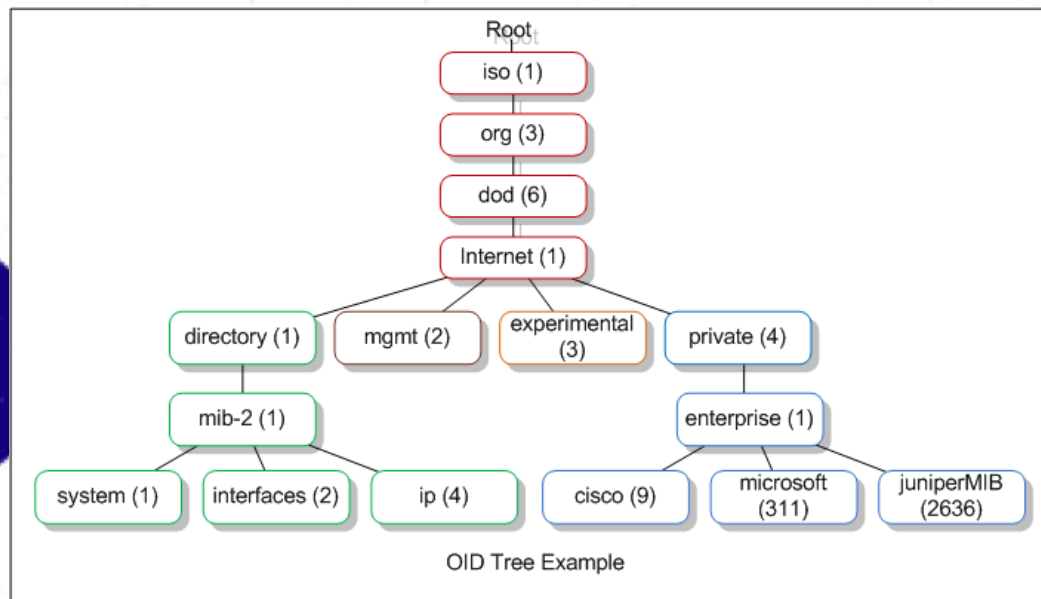
Schema Design contd.

- LDAP based schemas consist of:
 - Attribute types.
 - Attribute syntaxes.
 - Matching rules.
 - Object classes.

Attributes

- As we have discussed, in a DIT, the directory entries contain a collection of attribute types and values.
- Attribute types (or simply attributes) hold specific data elements such as a name, business phone number etc.





- Each attribute type is having a *definition*.
- The *definition* contains the following:
 - A name that uniquely identifies the attribute type.
 - An OID that also uniquely identifies the attribute.
 - A textual description.
 - An indication of whether the attribute is single or multi-valued.
 - An associated attribute syntax and set of matching rules.
 - An attribute usage indicator.
 - Restrictions on the range or size of the values that may be stored in the attribute.
 - Indication of whether the attribute can be modified by regular applications.

Attributes contd.

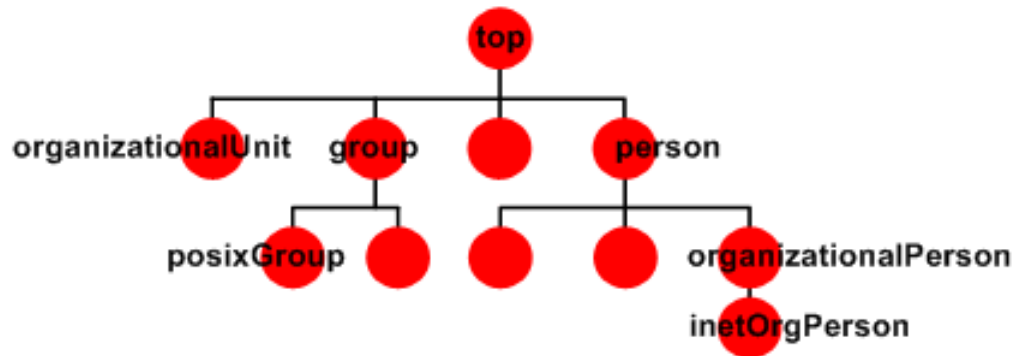
- Matching rules: govern comparisons and searches.
- Attribute usage indicator: whether for applications or for operation of the directory service itself.

Properties of Attributes

- They are not case sensitive.
 - Ex: cn and CN are the same.
- Only ASCII characters and the hyphen are allowed.
 - Valid: cn, sn, postal-Address.
 - Invalid: -cn, hello#there.
- Name must start with a character.
- Unique across entire directory.
- Applications generally refer to an attribute using its name, rather than its OID.
- Attributes have one or more attribute values associated with them.
- Operational attributes are used by the directory itself for system related purposes.
 - Ex: modifyTimeStamp: date/ time an entry was last modified.

Attribute Syntaxes and Matching Rules

- Each attribute type has an associated:
 - Attribute syntax that specifies exactly how data values are represented and how comparisons between values are made.
- The rules for making attribute value comparisons are called matching rules.



- Object classes are used to group related information (or attributes).
- Each directory entry belongs to one or more object classes.
- The names of the object classes to which an entry belongs are always listed as values for a special multi-valued attribute called *objectclass*.
- The set of object classes associated with an entry serves the following needs:
 - Determines which attribute types must be included in the entry.
 - Determines which attribute types may be included in the entry.

Object Classes contd.

- An object class holds the following information:
 - A name that uniquely identifies the class.
 - Ex: person, printer, etc.
 - A textual description.
 - An OID that uniquely identifies the class.
 - A set of mandatory (required) attribute types.
 - A set of allowed (optional) attribute types.
 - A kind (structural, auxiliary or abstract) which indicates how the class is used.

```
attributetype ( 2.5.4.20 NAME 'telephoneNumber'  
  DESC 'RFC2256: Telephone Number'  
  EQUALITY telephoneNumberMatch  
  SUBSTR telephoneNumberSubstringsMatch  
  SYNTAX 1.3.6.1.4.1.1466.115.121.1.50{32} )
```

Matching rules

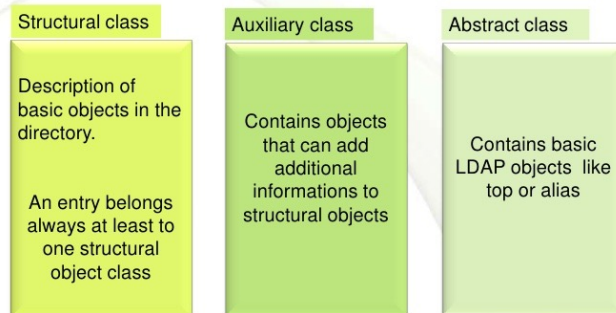
Type

Server should support values of this length

Table . . . Commonly Used Syntaxes

Name	OID	Description
boolean	1.3.6.1.4.1.1466.115.121.1.7	boolean value
directoryString	1.3.6.1.4.1.1466.115.121.1.15	Unicode (UTF-8) string
distinguishedName	1.3.6.1.4.1.1466.115.121.1.12	LDAP DN
integer	1.3.6.1.4.1.1466.115.121.1.27	integer
numericString	1.3.6.1.4.1.1466.115.121.1.36	numeric string
OID	1.3.6.1.4.1.1466.115.121.1.38	object identifier
octetString	1.3.6.1.4.1.1466.115.121.1.40	arbitrary octets

Three types of object class



Page • 21

- Structural classes describe the basic aspects of an object:
 - Most object classes are structural classes.
 - All entries should belong to exactly one structural object class.
- Auxiliary classes place no restrictions on where an entry may be stored:
 - Used to add a set of related attributes to an entry that already belongs to a structural class.
 - Sometimes call “mix-in” classes. Ex: simpleSecurityObject, mailRecipient, and cacheObject.
- Abstract classes may not be used directly, but only as ancestors of derived classes:
 - Rare. Abstract classes, the third and last kind of object class, are rare and are used only for classes needed to support LDAP's basic information model. Two examples of abstract classes are top and alias. Used only for classes needed to support LDAP's basic information model.
 - Ex: top, alias

Object Class Definitions...Examples

- objectclass (OIDp 'printer'

SUP top

STRUCTURAL

MUST cn

MAY (description \$ pagesPerMinute \$ languages)

)

- objectclass (OIDn 'networkDevice'

SUP top

AUXILLIARY

MUST ipaddress

MAY (cn \$ connectionSpeed)

)

Object Class Inheritance

- One object class can be derived from another: *inheritance*.
- The class that inherits some of its characteristics from is called the superior class or superclass.
- LDAP does not support multiple inheritance...!!!

Namespace Design

Your directory's namespace provides the basic means by which information is referenced in the directory

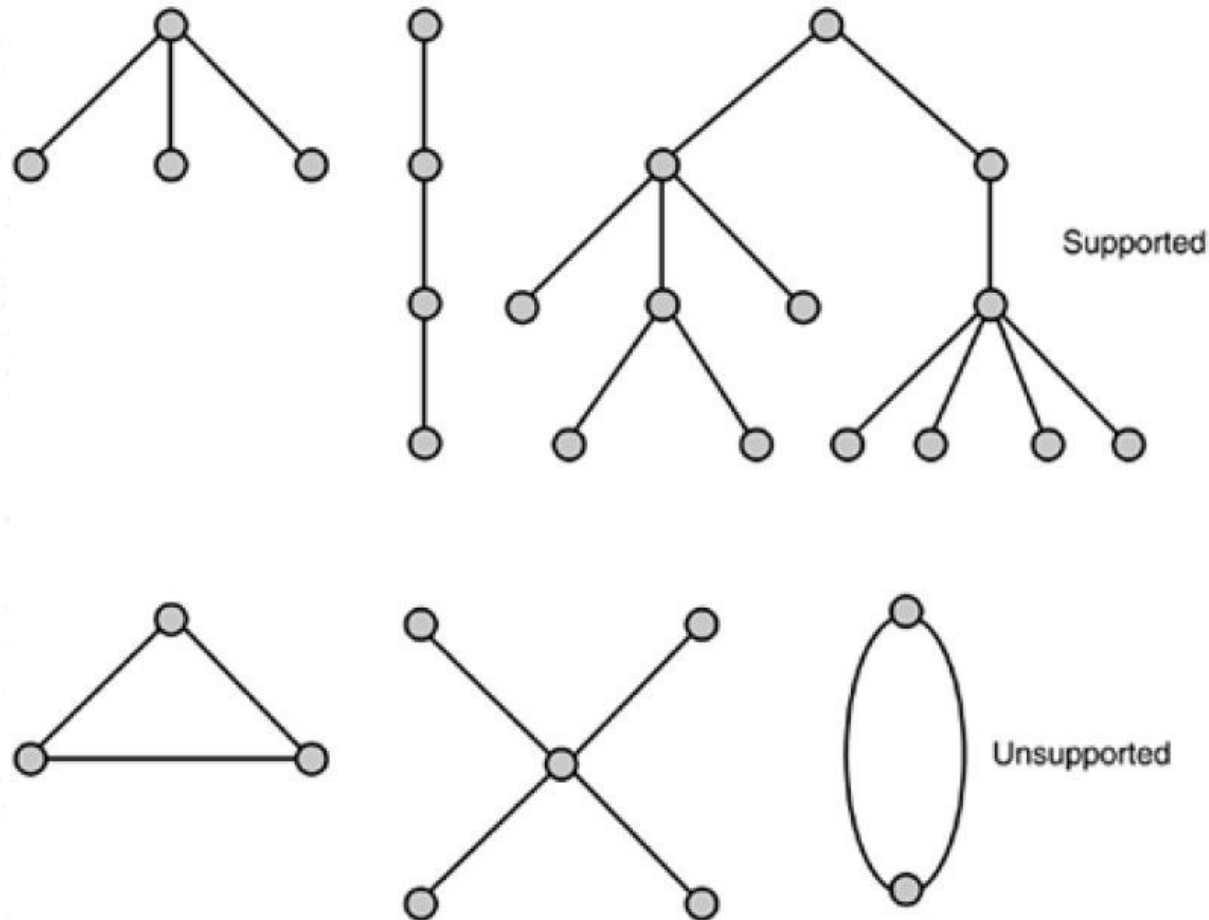
A properly designed namespace can lead to:

- Easier data maintenance.
- More flexibility in setting access control and replication policies.
- The ability to satisfy a wider variety of directory-enabled applications.
- More natural navigation through the directory.

- The LDAP model defines a flexible namespace framework, which means that you can almost certainly design a namespace to satisfy any requirements:
 - However, it also means that you have more choices to make than you might like.
- The LDAP namespace is inherited from the X.500 directory standard.
 - Tree-structured.
- The basic LDAP model is also hierarchical or tree-structured:
 - LDAP does not directly support namespaces in which an entry might be both a child and parent of the same entry:
 - alias and referral object classes can be used to construct such structures.
 - The seeAlso attribute indirectly supports such structures.



Examples of Supported and Unsupported Structures



Naming Entries in the Hierarchy

- Recall that an LDAP entry is a collection of attribute values.
 - The name of an entry is created by choosing one or more of these
 - attribute values to form a Relative Distinguished Name (RDN).
 - However, multi-valued RDNs are discouraged.

cn: Babara Jensen cn: Babs Jensen sn: Jensen title: Director uid: babs mail: babs@airius.com	cn=Babara Jensen uid=babs uid=babs+sn=Jensen mail=babs@airius.com
Entry	Examples of RDNs

Naming Entries in the Hierarchy cont.

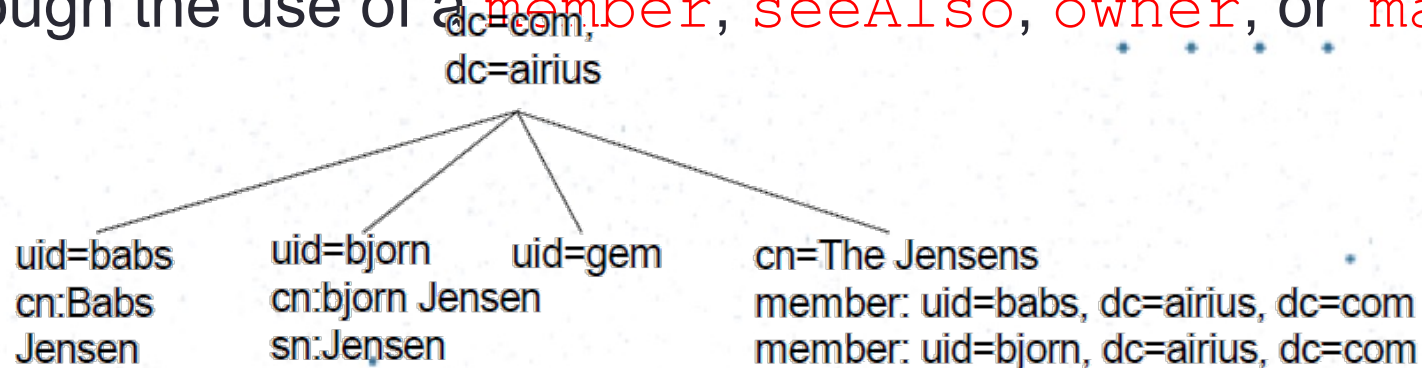
- When the RDN of an entry is chosen and its position relative to other entries in the hierarchy is determined,
 - forming the entry's full name is simple: Just combine the RDNs of the entry and all of its ancestor entries.
 - Resulting name is a Distinguished Name (DN).
 - DNs are written in little-endian order – Least significant component is written first.
 - E.g. cn=Barbara Jensen, o=Netscape, c=US

Purposes of a Namespace

Data Reference

A namespace provides the means by which **directory data is referenced**, which is important for two reasons.

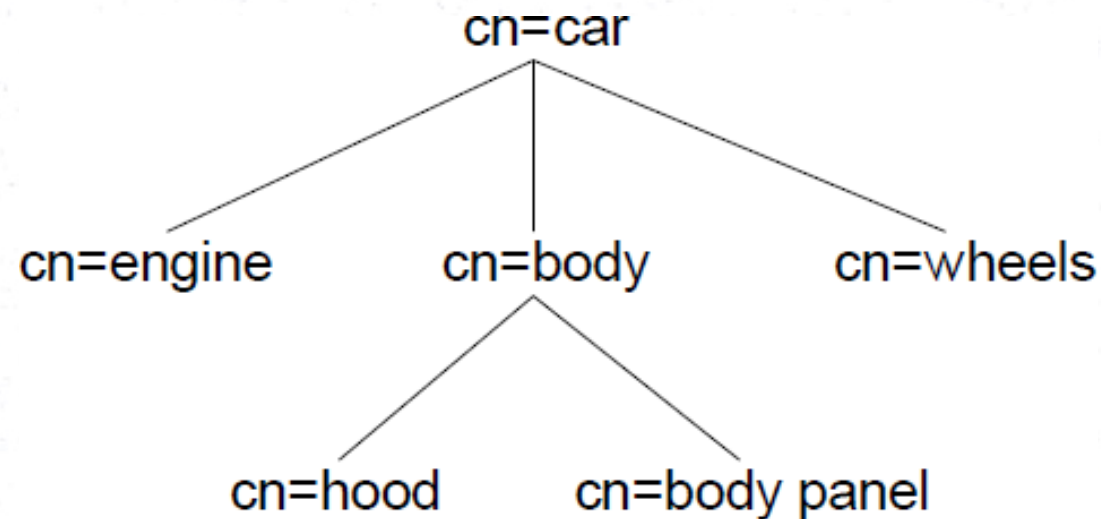
- First, there must be a way for directory clients to **refer unambiguously to a directory entry**.
- Second, the directory name provides a compact and efficient way to support groups of directory entries and directory entries that **refer to one another** (for example, through the use of a **member**, **seeAlso**, **owner**, or **manager** attributes).



Purposes of a Namespace cont.

Data Organization

For example, you might place all entries corresponding to people in one portion of the namespace, entries corresponding to devices in another etc. (further partitioning is possible).



Purposes of a Namespace cont.

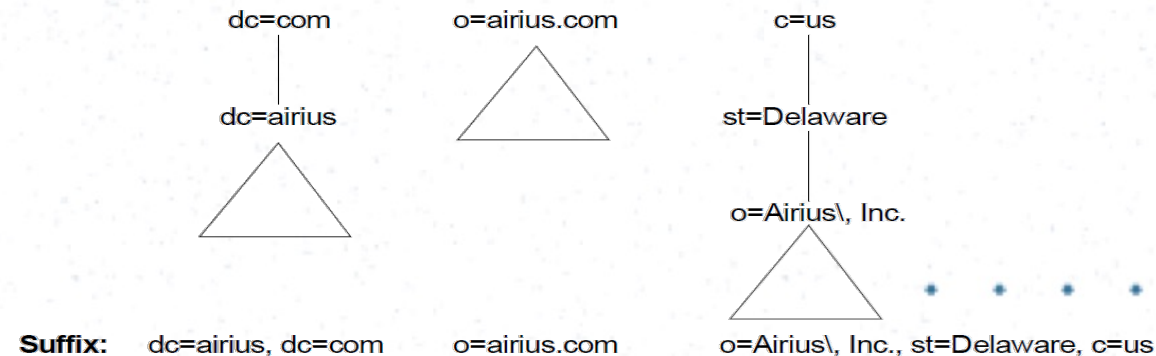
- **Data partitioning:**
 - ❑ A namespace enables directory data to be partitioned, or divided among multiple servers.
 - Partitioning can be performed only at a branch point in the directory.
- **Data replication:**
 - ❑ Choice of namespace design can constrain the replication scenarios your directory can support.
 - ❑ Most replication solutions require partitioning on branch points in the namespace.
- **Access control:**
 - ❑ Like replication, some products allow the setting of access control only at namespace branch points.
- **Application support:**
 - ❑ From a functional perspective, the namespace should support the applications that the directory is serving.

Analyzing Namespace Needs

- Now that you have some idea of what you're going to do with your namespace after you define it, it's time to turn our attention to the **design process itself**.
- **In this section we will look at the major areas and decisions you'll need to make when designing your namespace.**
 - ☐ Choice of suffixes.
 - ☐ Flat or hierarchical namespace.
 - ☐ What attributes should you use to name entries.
 - ☐ Application requirements.
 - ☐ Administrative concerns.
 - ☐ Privacy concerns.
 - ☐ Future needs.
- We'll try to figure out what you gain and what you sacrifice at each above step.

Choosing a Suffix

- The directory service may have only a local scope or it may be part of a larger or global directory system:
 - ❑ 3 methods are commonly used to choose the format of the suffix – RFC 2247, organization's DNS name itself or X.500 convention.
 - ❑ Directory's suffix should name the top of your department's tree otherwise.

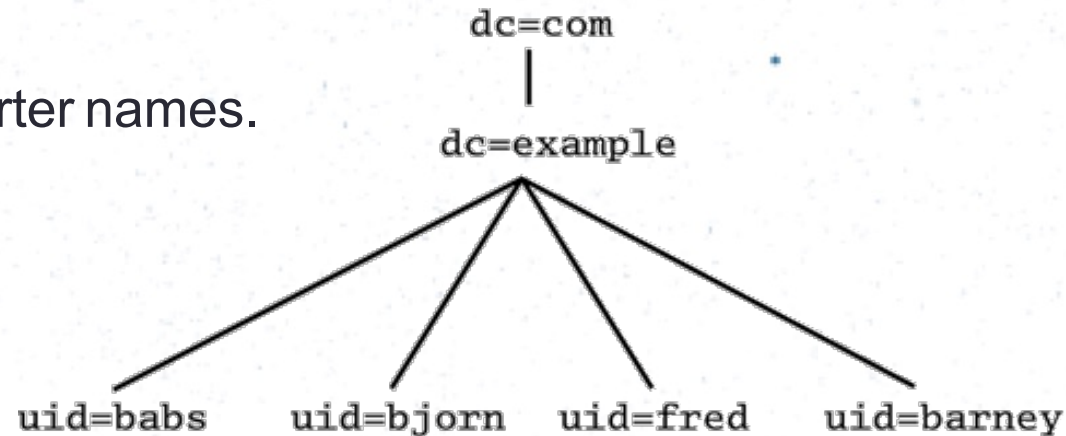


- A directory server may hold multiple suffixes:
 - ❑ You may want to design a service with multiple suffixes if you have two or more trees of information that do not have a natural common root.

Flat and Hierarchical Schemes

Flat Namespaces

- Less grouping and shorter names.

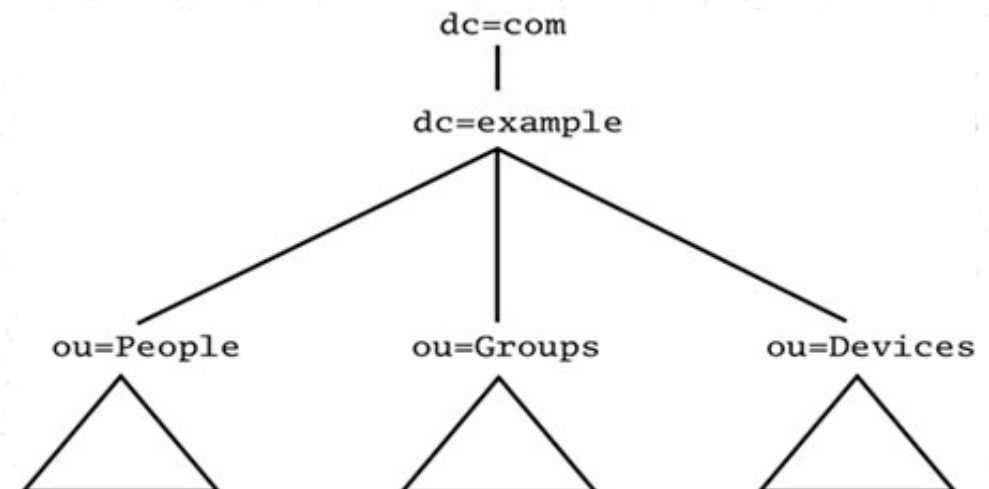


- As a guiding design principle, you should strive to mark a directory's namespace as flat as possible:
 - ❑ The flatter the namespace is, the **less likely names are to change**.
 - ❑ **Shorter names take up less space**.
 - E.g. Increasing the average name by only 20 bytes (the approximate result of adding an extra component) represents an increase of 2MB in a directory containing 100,000 entries..!
 - ❑ **Shorter names are easier to remember..!**

Flat and Hierarchical Schemes

Hierarchical Namespaces

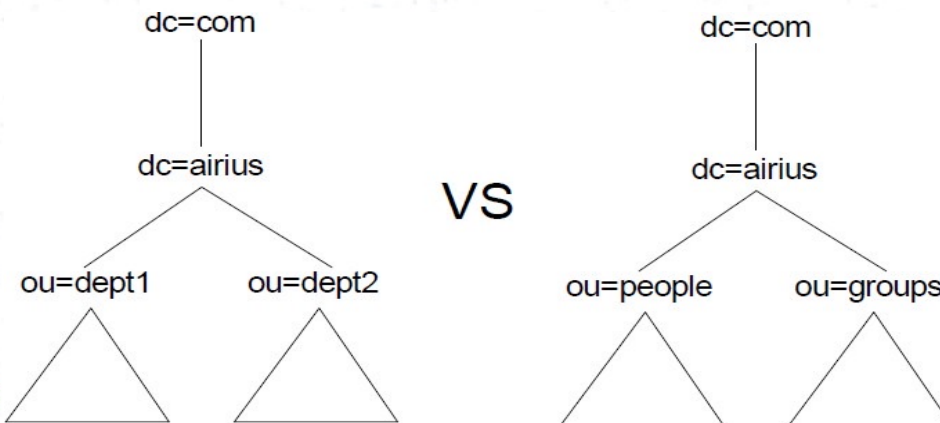
- Hierarchies can be used to **partition data amongst servers, enable replication, provide finer-grained access control, and browsing purposes.**
 - ❑ However, if a centralized directory is anticipated, there is no need to introduce hierarchies to enable data distribution.



Flat and Hierarchical Schemes

Hierarchical Namespaces cont.

- Design hierarchy in a way that **minimizes problematic name changes**.
- Name changes can be avoided if you are able to design your hierarchy based on information that is **not connected to directory information that is likely to change**.



Naming Attributes

What attributes should you use to name entries

- **Requirements** imposed by LDAP on **naming attributes**:
 - ❑ The RDN of the entry must be chosen from one or more of the entry's attributes.
 - ❑ The RDN of the entry must be unique among all its siblings (other entries that have the same parent).
- Although these are the only restrictions, a **policy** which ensures **unique RDNs for people** is preferable (*unique names for people across the entire directory*):
 - ❑ **Has the benefit that, even if an entry has to be moved, no name clashes will occur.**

Naming Attributes cont.

What attributes should you use to name entries

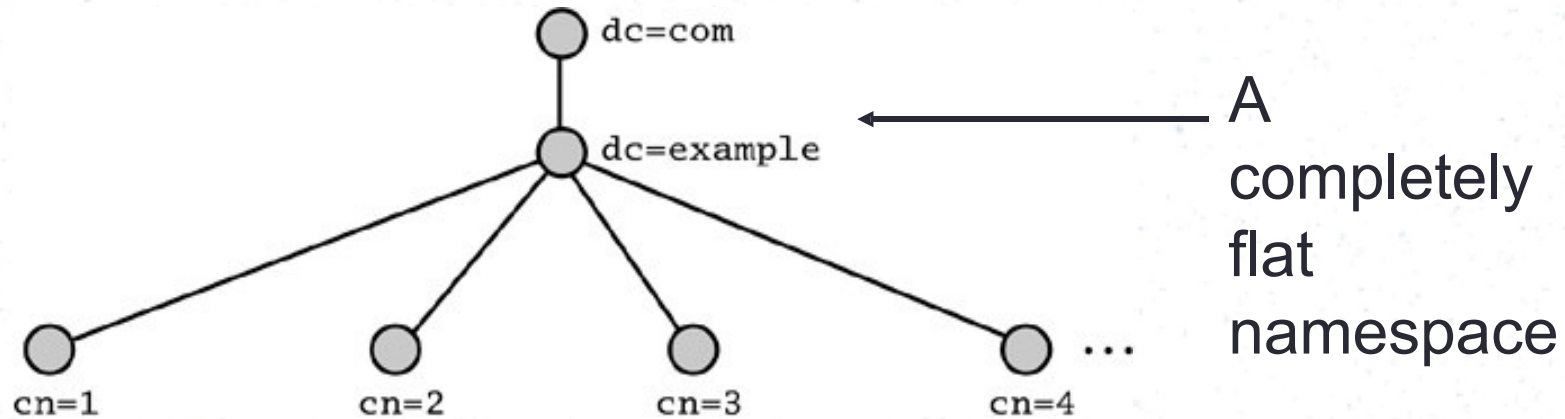
Two approaches to ensuring unique names (especially for people):

- **Using existing unique names:**
 - ❑ Employee IDs.
 - ❑ Unix login names.
- **Constructing new identifiers:**
 - ❑ Timestamping techniques.
 - ❑ Appending numbers to names.
 - ❑ **Multivalued RDNs (strongly discouraged).**

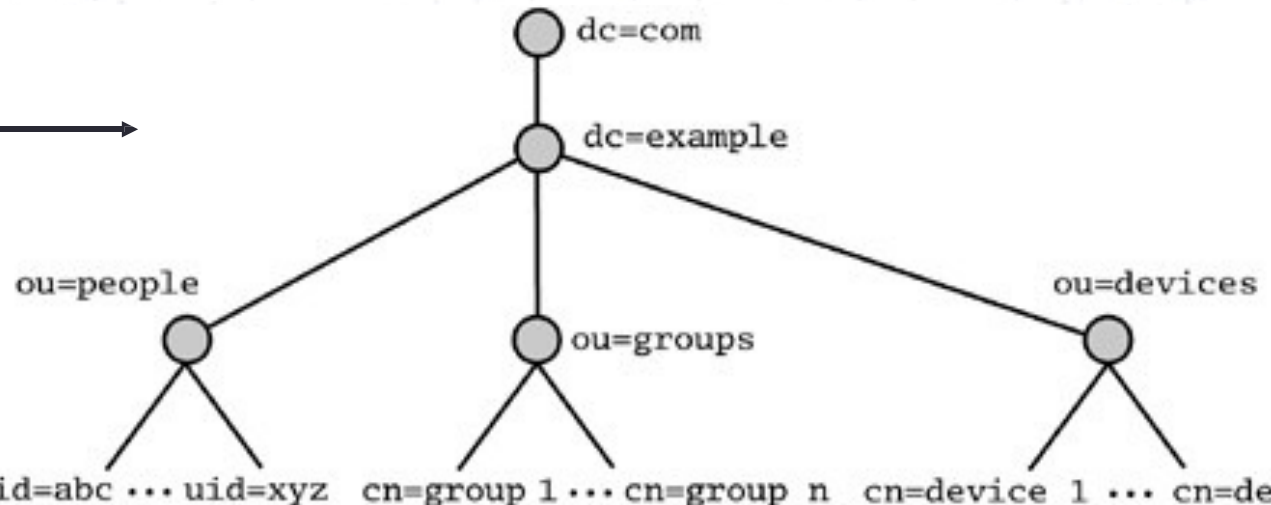
Other Considerations

- **Applications considerations**
 - ❑ Typically, a directory is required to support one or more directory enabled applications.
- **Administrative considerations**
 - ❑ What is the impact of your namespace design on administrative tasks, such as adding/deleting entries, name changes or migration across directory server platforms.
- **Privacy considerations**
 - ❑ You should not reveal any information through the namespace that you do not intend to reveal..!
- **Anticipate the future** (situations that precipitate a namespace redesign):
 - ❑ Choosing the wrong naming attribute.
 - ❑ A directory starts out under central administrative control, but you later decide to hand over some portion of the data.
 - ❑ If you choose a hierarchical namespace with a hierarchy based on a geographical, organizational or any scheme that is bound to change.

Some Example Namespaces

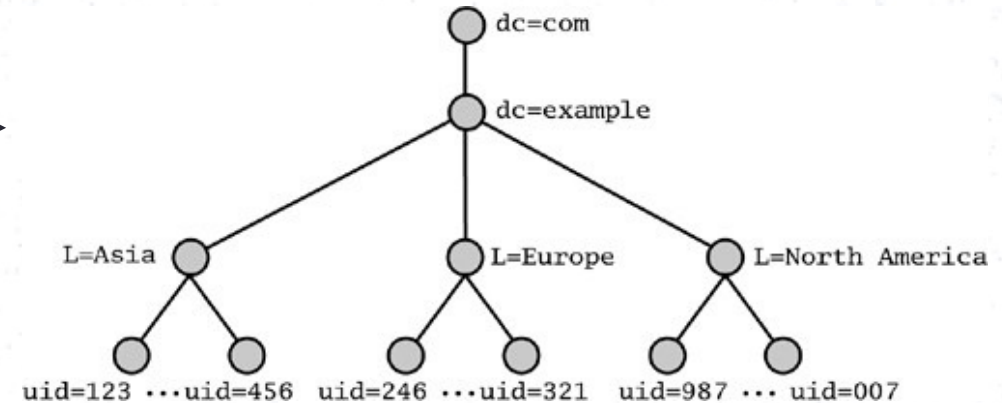
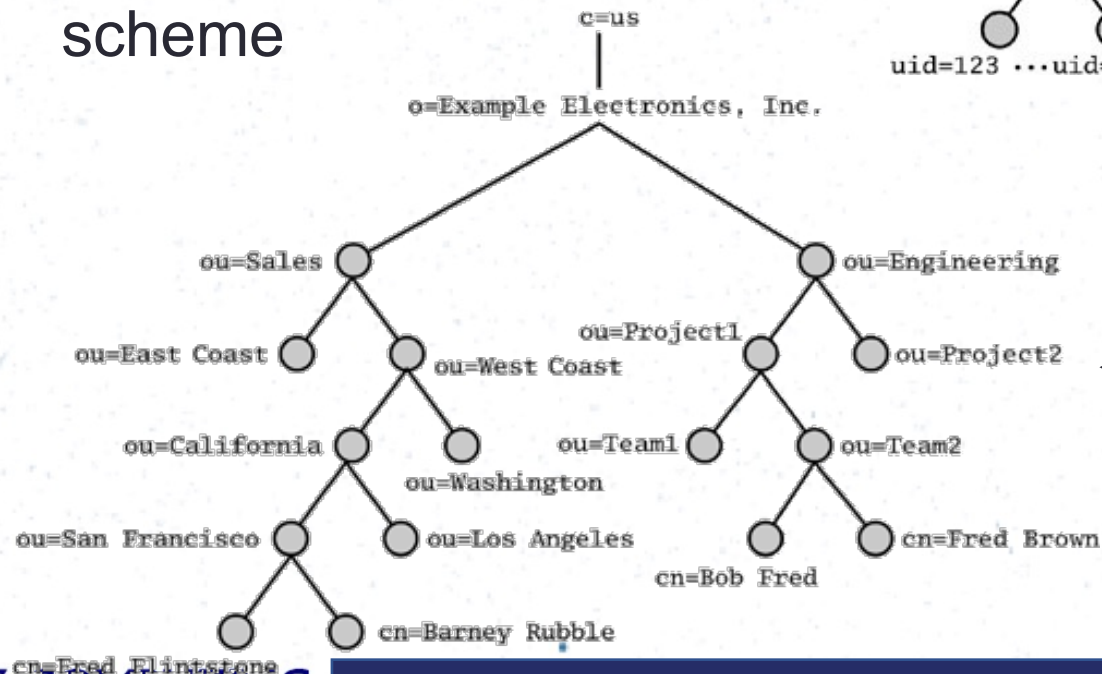


A flat namespace with some hierarchy



Some Example Namespaces

A hierarchical namespace based on a geographical scheme



A hierarchical namespace based on organizational structure

Topology Design

Topology Design

- Partitions
- Gluing partitions
 - Referrals
 - Chaining
- Name resolution
- Authentication in a distributed directory
- Directory partitioning in openLDAP

What is Topology Design?

- LDAP directory services are designed to support a distributed directory
- The directory's topology describes:
 - the way you divide your directory tree among physical servers and
 - How you allocate those servers among your organisation's physical locations
- Benefits:
 - Achieve optimal performance for directory enabled applications
 - Increase directory availability
 - Better managed directory



Directory Topology Overview

- When you divide a single directory into manageable chunks and assign them to separate servers, you are partitioning the directory:
 - The directory itself is responsible for hiding partitioning details from users
 - Each server is responsible for only a portion of a tree
 - DNS operates in the same manner
 - In X.500 terminology, the unit of division is known as a naming context , partition or suffix
 - A directory partition is a complete sub-tree of the DIT
 - All entries in a partition must share a common ancestor known as the partition root

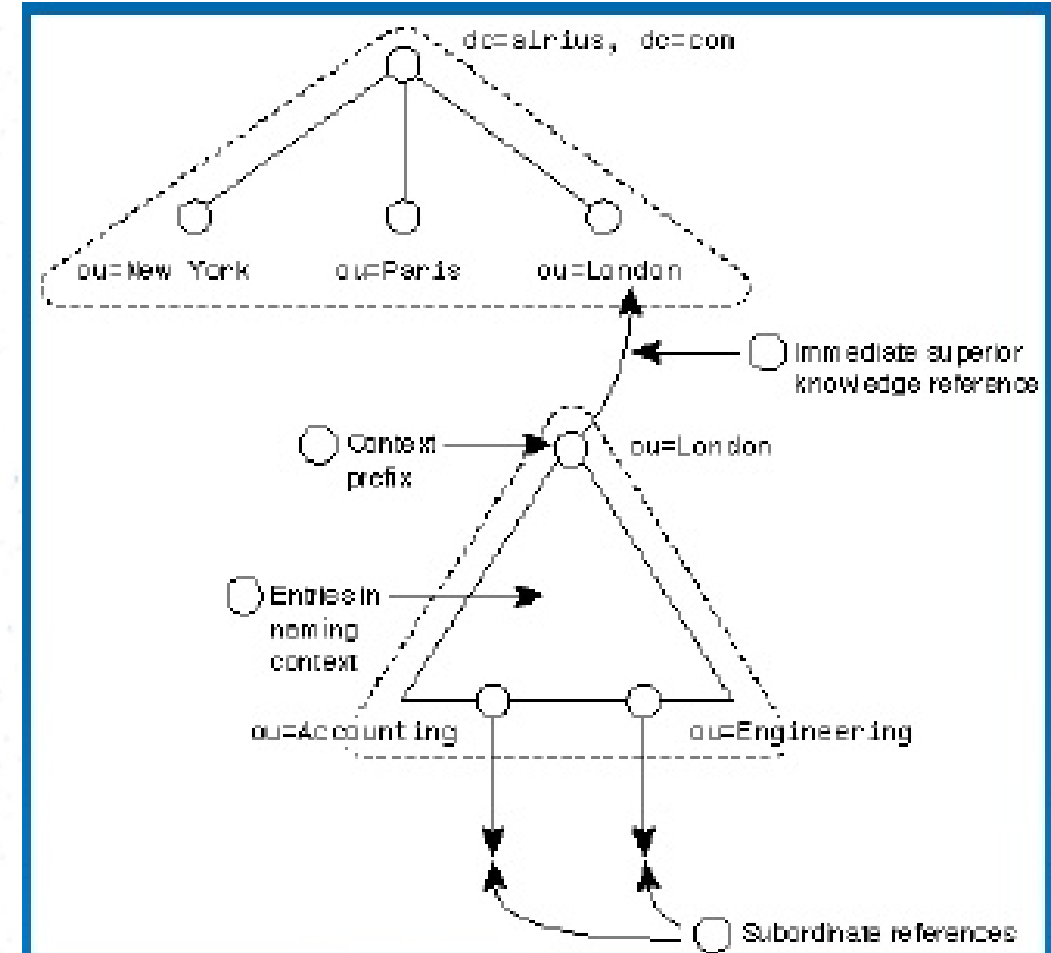


Directory Topology Overview

- It is also possible to exclude directories from a partition
- Using these principles you can divide a single large directory into a number of smaller partitions

Gluing the Directory Together

- We need some way to describe the relationships between directory partitions:
 - LDAP and X.500 defines these relationships in terms of knowledge references
- There are 2 types of knowledge references:
 - Immediate superior knowledge references: Point upward in the DIT toward the root and ties the naming context to its ancestor
 - Subordinate references: Point downward in the DIT to other partitions



Name Resolution

- Name resolution is the process by which a directory maps a DN provided by a client to an actual object in the directory
- Name resolution is used in the following circumstances:
 - When locating the base object of an LDAP search or compare operation
 - When locating an entry to modify, delete, rename or bind as
 - When locating the parent of an entry to be added to the directory
- A DN presented in this fashion is called a purported name:
 - The client claims that it exists and its up to the directory system to check whether this is true

Name Resolution (Example)

1. The client presents the purported name
2. Since server 1 does not hold a naming context that could contain the entry, the immediate superior knowledge reference allows the name resolution process to walk up the tree to server 2
3. Server 2 contains a subordinate reference for the naming context ou=engineering, dc=airius, dc=com, so the name resolution process goes to server 3
4. Server 3 holds a subordinate reference for the naming context ou=PCB design, ou=engineering, dc=airius, dc=com, so the process walks down again
5. Finally, server 4 is consulted to check whether the entry exists

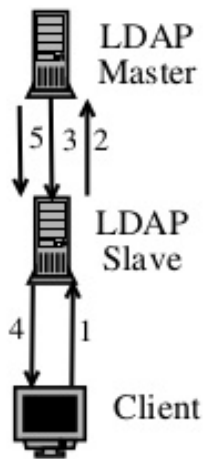
Handling Distribution in the Client

- LDAP referrals and search result continuation references are pieces of knowledge reference information sent from an LDAP server to an LDAP client:
 - Indicates that other servers need to be contacted to fulfil request
- LDAP referrals – When client performs add, modify, delete, moddn, compare or search operation with a base-level scope and the server does not hold target entry, an LDAP referral is returned
- Search result continuation references – When search scope is not base-level and the server has knowledge of other subordinate directory partitions, search result continuation references are returned
- Similar – Whereas referrals are contained in LDAP result messages, search result continuation references are contained in search result messages
- Normally, client libraries automatically handle processing of references

Structure of a LDAP Referral

- The information in a LDAP referral is in the format of a LDAP Uniform Resource Locator (URL)
- A referral gives the following information:
 - The hostname of the server to contact
 - The port number of the server
 - The base DN (search operation) or target DN (add, delete, compare or moddn operation)
 - Optional: if absent, the client should use the same base DN it used when performing the operation that resulted in the referral
- E.g. if a client searches the subtree dc=airius, dc=com for all entries with a surname smith, the referral would be returned as the following LDAP URL:

`ldap://server3.airius.com:389/ou=engineering, dc=airius, dc=com`



1. Client sends modification to replica
2. Replica forwards request to master
3. Master returns result to replica
4. Replica forwards result to client
5. Master updates replica

Handling Distribution in the Server - Chaining

- When chaining is used, the client starts by submitting an operation to a server in the usual way:
 - If the server is unable to completely process the request because it does not hold all the required data, it chases the referral by contacting other servers on behalf of the client
 - It returns the combined results to the client when the other servers have completed the operation

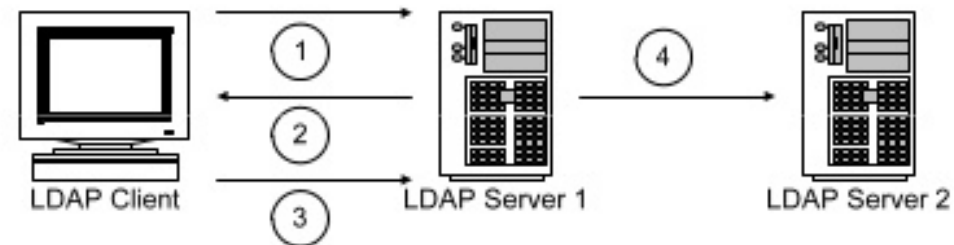
Deciding Between Client or Server Distribution Handling

- Chaining in general reduces client complexity, but at the cost of increased server complexity:
 - Opposite is true if client is required to handle references
- You may not have a choice about which method you use:
 - openLDAP only supports client-side processing



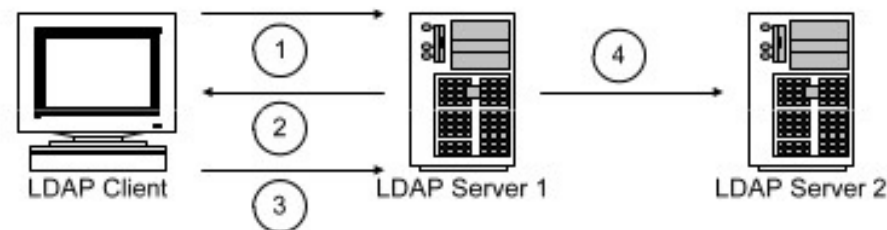
Authentication in a Distributed Directory

- The server or servers that ultimately handle a client request must verify the identity of the client so that they can enforce restrictions
 - This is true even if the server handling the request is not the server to which the client originally authenticated
- Authentication in a chained environment is accomplished as follows:
 1. The directory client connects to server 1 and authenticates
 2. If the client's authentication credentials are successfully verified, server 1 returns a success code to the client
 3. The client submits a search operation to server 1
 4. Server 1 determines that it does not hold the appropriate partition, so it chains the operation to server 2



Authentication in a Distributed Directory

- There are 2 ways server 2 might learn the clients identity:
 - Server 1 might tell server 2 who the client is and server 2 might choose to believe server 1
 - Server 1 might pass on to server 2 the client's identity and authentication credentials:
 - Server 2 could then independently verify the credentials



Security Implications

- When using referrals, you should take precautions to ensure that you have absolute control over the referrals contained in your directory:
 - Additionally, you should allow referrals only to directories you trust
- Important because some directory clients may choose to automatically resubmit referred operations and authenticate to the referred-to server using the same credentials used when authenticating to the original server:
 - This means that if a rouge referral were placed in a directory, the directory clients may be tricked into submitting their authentication credentials
 - Similar principles apply for chaining – only trust remote servers under your direct control



When to Partition?

- Factors favouring a directory with larger number of partitions:
 - The number of entries is too great for a single partition or server
 - Your directory-enabled applications tend only to read and modify entries from the local workgroup
 - The amount of update traffic to a single partition is too large
 - Partitioning allows data and update traffic to remain local and avoid spanning WAN links
 - Directory use will expand significantly in the future, beyond the point where a single partition is feasible



Replication Design

Replication

- Replication design
- Replication scope
- Consistency and convergence
- Replication strategies



Replication Design

- Replication increases the availability of your directory server:
 - Increased reliability - Protects your service from equipment failures, network partitioning etc...
 - Improved read performance – Reduces server, network load, thus reducing latency at clients and better load distribution
 - Improved write performance – Multi-master replication
- Security purposes:
 - By selectively replicating portions of a directory, the replica could function as a public directory service



Replication Concepts

- Suppliers, consumers and replication agreements
- The unit of replication
- Consistency and convergence
- Incremental and total updates
- Initial population of a replica
- Replication strategies

Suppliers, Consumers and Replication Agreements

- The terms supplier and consumer refer to the source and destination of replication updates:
 - These roles are not mutually exclusive
- The configuration information that tells a supplier server about a consumer server (and vice versa) is termed a replication agreement:
 - includes the unit of replication
 - the hostname and port of the remote server and
 - scheduling information
- In essence, the replication agreement defines WHAT is to be replicated, WHERE it is to be sent and HOW it is to be done



The Unit of Replication

- Define subtree – specify DN at the top of a subtree and replicate all entries subordinate to it
- Filtered replication (unsupported by openldap):
 - Define select entries in subtree – based on objectclass definitions:
 - X.500 defines this ability as the specification filter component of the unit replication
 - Define select attribute types – based on attributetype definitions:
 - X.500 defines this as the attribute selection component of the unit of replication

Consistency and Convergence

- Consistency describes how closely the contents of replicated servers match each other at a given point in time:
 - A strongly consistent replica is one that provides the same information as its supplier at all times
 - A weakly consistent replica is permitted to diverge from its supplier for some period of time:
 - All practical directory systems use weakly consistent replicas
- We say that a supplier and consumer have converged when they contain the same data



Initial Population of a Replica

- The consumer (replica) contains no data when it is initially configured:
 - Or, in the event that a replica becomes damaged, it must be brought back into synchronisation
- Directory vendors accomplish replica initialisation through similar techniques:
 - X.500 Directory Information Shadowing Protocol supports a total update strategy
 - Most LDAP products, including openLDAP, use LDAP itself to initialise the replica:
 - sending a series of delete operations to remove undesired entries and a series of add operations to populate the directory

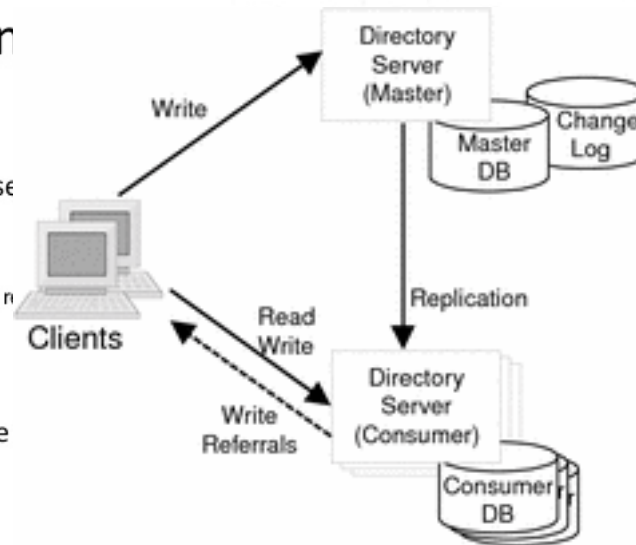
Replication Strategies

- The term replication strategy refers to the way updates flow from server to server and the way servers interact when propagating updates
- But after a client has successfully modified, deleted, added or renamed an entry, how does the server make it visible to its replicas?
- There are 3 main approaches toward solving this problem:
 - Single-master replication
 - Floating-master replication (unsupported by openldap) and
 - Multi-master replication (unsupported by openldap)



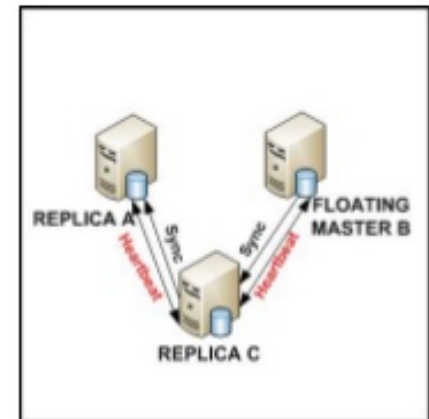
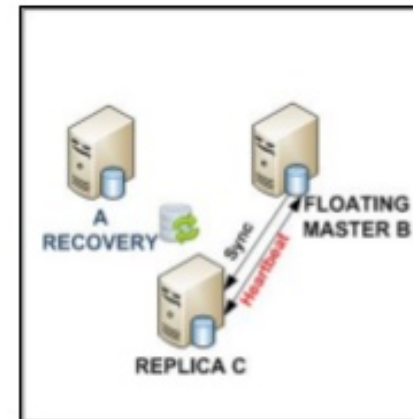
Single Master Replication

- In single-master replication, there is one and only one server that contains a writable copy of a given directory entry.
 - Replicas are all read-only.
 - Advantage: since directory applications in general perform more read operations.
 - Disadvantage: single point of failure as the read-write server.
- What if the client needs to write to the directory via the replicas?
 - Referrals
 - Chaining



Floating-Master Replication

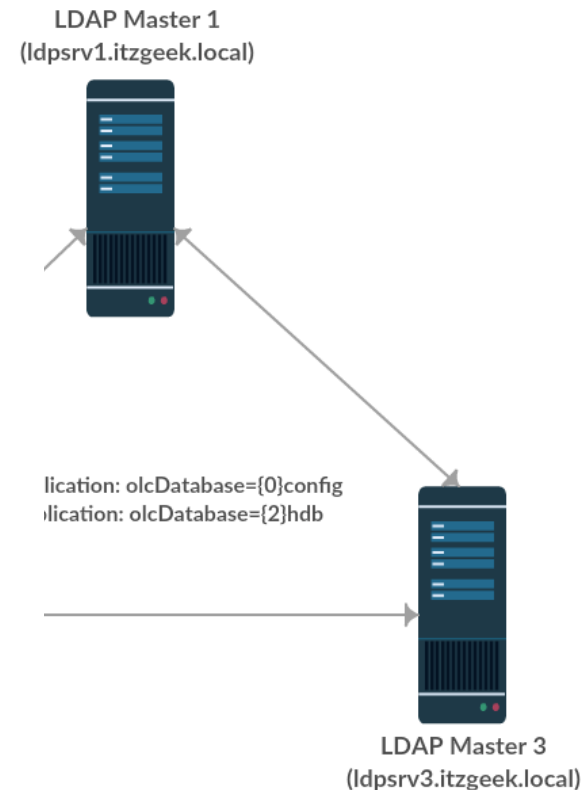
- Like single-master replication, floating-master replication maintains only a single read-write server:
 - When the read-write server becomes unavailable, a voting-based algorithm is used to elect a read-write master from the remaining replicas
 - Avoids single-point of failure
- Complications in synchronisation arise when 2 masters are rejoined:
 - Requires an update conflict resolution policy
 - **Last writer wins policy** - timestamps
 - Business rules – precedence between departments
 - Never used in directory products



Multi-Master Replication

Multi-master Replication

- In multi-master replication, there may be more than one read-write server available:
 - Clients submit a write operation to any of the read-write replicas.
 - Improves write performance
 - It becomes the responsibility of the set of cooperating servers to ensure that changes are eventually propagated
- Like floating master replication, this approach requires an update conflict resolution policy



LDAP Multi-Master Replication

Thank You